



Bachelor-Thesis

Entwicklung einer Observability-Strategie für das Audience Response System "frag.jetzt" basierend auf dem Konzept der Four Golden Signals.

zur Erlangung des akademischen Grades

Bachelor of Science

im dualen Studiengang Informatik an der IU Internationale Hochschule

von

Max Mustermann

Matrikelnummer: 123401442

28. März 2026

Referent: Prof. Dr. Klaus Quibeldey-Cirkel

Korreferent: Prof. Dr. Philipp Diebold

Kurzfassung

Das Abstract besteht normalerweise aus einem Absatz, welcher die Hauptziele, Resultate und Schlussfolgerungen der Thesis zusammenfasst. Es ist auf Deutsch und Englisch zu verfassen; es soll ungefähr 200 Wörter (in jeder Sprache) beinhalten und insgesamt nicht über eine Seite lang sein. Des Weiteren sollten Schlüsselwörter unter diesen Absatz geschrieben werden. Schlüsselwörter sind drei bis sieben Wörter, die die Leserschaft das Thema erkennen lassen.

Abstract

Das Abstract besteht normalerweise aus einem Absatz, welcher die Hauptziele, Resultate und Schlussfolgerungen der Thesis zusammenfasst. Es ist auf Deutsch und Englisch zu verfassen; es soll ungefähr 200 Wörter (in jeder Sprache) beinhalten und insgesamt nicht über eine Seite lang sein. Des Weiteren sollten Schlüsselwörter unter diesen Absatz geschrieben werden. Schlüsselwörter sind drei bis sieben Wörter, die die Leserschaft das Thema erkennen lassen.

Danksagung

An dieser Stelle möchte ich meinen aufrichtigen Dank aussprechen.

Zuallererst danke ich meinen Betreuern **Prof. Dr. Klaus Quibeldey-Cirkel** und **Prof. Dr. Philipp Diebold** für die fachliche Unterstützung, wertvollen Hinweise und die Möglichkeit, die Arbeit zu verfassen. Ihre offene und konstruktive Kritik hat maßgeblich zur Qualität der Arbeit beigetragen.

Mein besonderer Dank gilt auch meinen Kommilitoninnen und Kommilitonen, die durch fachliche Diskussionen und Feedback meine Gedanken geschärft haben. Ihre Perspektiven haben mir geholfen, verschiedene Aspekte der Thematik tiefergehend zu beleuchten.

Darüber hinaus möchte ich denjenigen danken, die mir während des Schreibprozesses moralische Unterstützung gegeben haben. Ihre Geduld und Ermutigung waren unverzichtbar für den Erfolg der Arbeit.

Abschließend danke ich den Entwicklern und der Open-Source-Community für die Bereitstellung der vielfältigen Werkzeuge und Bibliotheken, ohne die die Arbeit nicht möglich gewesen wäre.

Hinweis zur geschlechtergerechten Sprache

In dieser Arbeit wird überwiegend eine geschlechtsneutrale Sprache verwendet. Aus Gründen der besseren Lesbarkeit wird in einzelnen Fällen das generische Maskulinum genutzt. Selbstverständlich sind damit Personen aller Geschlechter gleichermaßen gemeint.

Alle verwendeten Personenbezeichnungen gelten gleichermaßen für alle Geschlechter. Die Arbeit orientiert sich an den Empfehlungen des Leitfadens zur gendersensiblen und inklusiven Sprache der IU Internationale Hochschule.

Inhaltsverzeichnis

Kurzfassung	i
Abstract	ii
Danksagung	iii
Hinweis zur geschlechtergerechten Sprache	iv
Abbildungsverzeichnis	ix
Tabellenverzeichnis	x
Listings	xi
Abkürzungsverzeichnis	xii
1 Einleitung	1
1.1 Forschungsfragen	2
2 Theoretische Fundierung	3
2.1 Abgrenzung von Monitoring und Observability in verteilten Systemen	3
2.2 Besonderheiten verteilter und cloud-nativer Anwendungen	3
2.3 Telemetriearten und ihr Zusammenspiel: Logs, Metrics und Traces	5
2.4 OpenTelemetry als Standard für herstellerübergreifende Instrumentierung	5
2.5 Die Four Golden Signals als heuristischer Bezugsrahmen	6
2.6 SLIs, SLOs und Error Budgets	7
2.7 Alert Fatigue, Actionable Alerts und Runbooks	7
3 Systemanalyse und Anforderungsdefinition: „frag.jetzt“	9
3.1 Fachlicher Kontext und Nutzungsszenario	9
3.2 Technische Architektur	9
3.3 Kritische User Journeys	11
3.3.1 Journey 1: Frage stellen.	11
3.3.2 Journey 2: Live-Voting.	11
3.3.3 Journey 3: KI-Zusammenfassung.	11
3.4 Risikopunkte und Zuordnung zu den Golden Signals	11
3.4.1 R1: Fehlende Timeouts und Resilienz.	11
3.4.2 R2: Datenbankengpässe bei Massenabstimmungen.	11
3.4.3 R3: Backend als Single Point of Failure ohne Observability.	12
3.4.4 R4: WebSocket-Verbindungsverlust.	12

3.4.5	R5: Fehlende Ressourcenlimits.	12
3.4.6	R6: Single-Host-Topologie als systemweiter Ausfallpfad.	12
3.5	Observability-Ist-Zustand	12
3.6	Ableitung der Observability-Anforderungen	13
3.6.1	A1: Ende-zu-Ende-Nachvollziehbarkeit über Dienstgrenzen.	13
3.6.2	A2: Mehrstufige Messbarkeit entlang der Golden Signals.	13
3.6.3	A3: Strukturiertes, korrelierbares Logging.	13
3.6.4	A4: Handlungsfähige Alarmierung.	13
3.6.5	A5: Beobachtbarkeit der Infrastrukturschicht.	14
4	Methodisches Vorgehen und Bewertungsrahmen	15
4.1	Arbeitslogik der Arbeit	15
4.2	Vorgehen bei der Ableitung der Golden Signals pro Service	15
4.3	Kriterien für Stack-Auswahl und spätere Evaluation	16
4.3.1	Vergleichskriterien für die Stack-Auswahl	16
4.3.2	Bewertungskriterien für die Gesamtstrategie	17
5	Evaluation und Auswahl des Observability-Stacks	18
5.1	OpenTelemetry als verbindliche Instrumentierungsschicht	18
5.2	Vergleich der Zielarchitekturen	18
5.2.1	Elastic Observability	18
5.2.2	Grafana-Stack: Prometheus + Loki + Tempo + Grafana	19
5.2.3	Prometheus + Jaeger + Loki + Grafana	19
5.3	Vergleich und Synthese	19
5.4	Begründete Zielentscheidung für frag.jetzt	20
5.4.1	Entscheidungsbegründung im Kontext von frag.jetzt	20
5.4.2	Kompromisse und Einschränkungen	21
6	Konzeption der Observability-Strategie für frag.jetzt	22
6.1	Leitprinzipien der Strategie	22
6.2	Logische Zielarchitektur der Observability	22
6.2.1	Instrumentierungsschicht	23
6.2.2	Sammel- und Verarbeitungsschicht	23
6.2.3	Speicherschicht	23
6.2.4	Analyse- und Visualisierungsschicht	24
6.2.5	Einordnung im Kontext von frag.jetzt	24
6.3	Servicebezogene Operationalisierung der Four Golden Signals	25
6.3.1	PWA-Frontend	26
6.3.2	Spring-Boot-Backend	26
6.3.3	PostgreSQL	26

6.3.4	Externe KI-Dienste	27
6.4	Serviceübergreifende Signal-Matrix	27
6.5	Rollenbasiertes Informations- und Dashboard-Konzept	29
6.6	Alerting-Konzept und Runbook-Logik	29
6.6.1	Alarmkategorien	29
6.6.2	Runbook-Verknüpfung	30
7	Prototypische Implementierung	31
7.1	Umfang und Abgrenzung der prototypischen Umsetzung	31
7.2	Instrumentierung ausgewählter Kernkomponenten	31
7.2.1	Observability-Stack und Telemetrie-Pipeline	31
7.2.2	Applikationsinstrumentierung über den OTel Java Agent	32
7.2.3	Infrastrukturexporter und Scrape-Konfiguration	32
7.2.4	Logging-Anbindung	33
7.2.5	Exemplarische Validierungsszenarien	33
7.3	Umsetzung des Golden-Signals-Dashboards	34
7.4	Alerting-Regeln und Beispiel-Runbooks	35
7.5	Herausforderungen und Lösungen bei der Umsetzung	36
7.5.1	Ressourcendimensionierung	36
7.5.2	Netzwerktopologie und Dienstkommunikation	36
7.5.3	Signalqualität und Rauschunterdrückung	36
8	Evaluation der entwickelten Strategie	38
8.1	Evaluationsmethodik	38
8.2	Anforderungsbezogene Bewertung	38
8.2.1	A1: Ende-zu-Ende-Nachvollziehbarkeit	39
8.2.2	A2: Mehrstufige Messbarkeit entlang der Golden Signals	39
8.2.3	A3: Strukturiertes, korrelierbares Logging	40
8.2.4	A4: Handlungsfähige Alarmierung	40
8.2.5	A5: Beobachtbarkeit der Infrastrukturschicht	41
8.3	Übergreifende Bewertungsdimensionen	42
8.4	Grenzen und methodische Einschränkungen	42
9	Diskussion	44
9.1	Einordnung der Ergebnisse im Verhältnis zur Literatur	44
9.2	Übertragbarkeit auf ähnliche cloud-native Plattformen	45
9.3	Wissenschaftlicher und praktischer Beitrag	45
	Literaturverzeichnis	46
	Index	48

A	Verzeichnis der KI-Nutzung	49
A.1	Copilot Prompts für die unterstützende Systemanalyse von frag.jetzt	50
A.2	C4-Container-Diagramm in Vollansicht	52

Abbildungsverzeichnis

2.1	Gegenüberstellung von lokalem Komponentenmonitoring und durchgängiger Anfrageverfolgung über mehrere Dienste (Distributed Tracing).	4
3.1	C4-Container-Diagramm der frag.jetzt-Architektur mit Applikationsdiensten, Infrastrukturkomponenten und Kommunikationswegen (Vollseitenansicht des Diagramms in Anhang A.2).	10
6.1	Logische Zielarchitektur der Observability-Pipeline fuer frag.jetzt. Das Schichtenmodell zeigt den Datenfluss von der Instrumentierung ueber Sammlung und Speicherung bis zur Auswertung. Gestrichelte Querverbindungen kennzeichnen Korrelationsmechanismen zwischen den Speicher-Backends.	25
7.1	Span-Waterfall eines Backend-Traces (<code>POST /livepoll/find</code>) mit 13 Spans und 15,4ms Gesamtdauer.	33
7.2	Service Graph aus Tempo mit den Kommunikationspfaden <code>user → fragjetzt-backend → fragjetzt-postgres</code> sowie <code>user → fragjetzt-ws-gateway</code>	34
7.3	Ausschnitt des Golden-Signals-Dashboards mit den Sektionen <i>Latency</i> und <i>Traffic</i>	35
7.4	Ausschnitt der <i>Errors</i> -Sektion mit Dummy-Fehlerrate zur Veranschaulichung der Fehlerindikatoren.	35
8.1	Trace-Waterfall in Tempo für den Endpunkt <code>GET /room/{id}/moderator</code> mit 11 Spans und zugeordneten Datenbankoperationen.	39
8.2	Log-to-Trace-Korrelation in Loki: Ein Logeintrag mit klickbarem TraceID-Link (links) navigiert zum zugehörigen Trace-Waterfall in Tempo (rechts).	40
8.3	Latenz-Alert im Zustand <i>Firing</i> nach 6 Minuten. Die Annotations enthalten Symptombeschreibung, Prüfpfad und Runbook-Link.	41
8.4	Infrastruktur-Sektion des Golden-Signals-Dashboards mit PostgreSQL-Verbindungen, RabbitMQ-Queue-Tiefe, Host-Ressourcen und Service Graph.	42
A.1	C4-Container-Diagramm der frag.jetzt-Architektur in Vollseitenansicht	53

Tabellenverzeichnis

3.1	Applikationstragende Dienste und ihre Observability-Relevanz	10
3.2	Zuordnung der Risikopunkte zu Golden Signals und Nutzerwirkung.	12
5.1	Vergleichende Bewertung der drei Observability-Zielarchitekturen	20
6.1	Serviceübergreifende Signal-Matrix mit Zuordnung von Dienst, Golden Signal, konkreter Messgröße, Erhebungszweck und primärer Stakeholder-Rolle.	28
7.1	Übersicht der instrumentierten Komponenten und beobachteten Telemetriesignale.	32
7.2	Zuordnung der prototypischen Umsetzung zu den Observability-Anforderungen.	37
8.1	Ergebnissynthese der anforderungsbezogenen Evaluation.	38
A.1	Verzeichnis der in dieser Arbeit genutzten KI-Werkzeuge	51

Listings

A.1 Copilot-Prompts zur Architekturanalyse 50

Abkürzungsverzeichnis

Abkürzung	Bedeutung
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
CPU	Central Processing Unit
ELK	Elasticsearch, Logstash, Kibana
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
KI	Künstliche Intelligenz
NLP	Natural Language Processing
OTel	OpenTelemetry
PWA	Progressive Web App
Q&A	Question and Answer
REST	Representational State Transfer
SLI	Service Level Indicator
SLO	Service Level Objective
SPA	Single-Page Application
SRE	Site Reliability Engineering
UI	User Interface
ARS	Audience Response System
FF	Forschungsfrage
TSDB	Time Series Database
REST	Representational State Transfer

1 Einleitung

Webbasierte Anwendungssysteme werden heute häufig als verteilte, cloud-native Anwendungen umgesetzt, die aus mehreren lose gekoppelten Services bestehen. Diese Architekturform erlaubt eine unabhängige Weiterentwicklung einzelner Komponenten sowie eine flexible Skalierung, erhöht jedoch zugleich den Aufwand für Betrieb, Überwachung und Fehleranalyse im Produkivsystem Faseeha et al., 2025; Sakinala, 2025. Insbesondere bei Microservices entstehen verteilte Fehlerzustände und komplexe Abhängigkeiten zwischen Diensten, wodurch Ursachen von Störungen nicht mehr eindeutig einzelnen Komponenten zugeordnet werden können Fischer et al., 2024.

Vor diesem Hintergrund gewinnt Observability als Erweiterung klassischer Monitoring-Konzepte an Bedeutung. Ziel von Observability ist es, den internen Zustand eines Systems anhand extern erfassbarer Anzeichen nachvollziehen zu können. Dazu werden Metriken, Logs und Traces kombiniert, um Zusammenhänge zwischen Systemverhalten, Fehlern und Leistungsmerkmalen sichtbar zu machen Faseeha et al., 2025. Empirische Untersuchungen aus dem Bereich cloud-nativer Anwendungen zeigen, dass ein strukturierter Einsatz von Observability-Praktiken die Analyse von Incidents unterstützt und die Reaktionszeit im Betrieb verkürzen kann Giamattei et al., 2024; Sakinala, 2025.

Ein häufig verwendeter Orientierungsrahmen zur Strukturierung servicebezogener Beobachtungsdaten sind die von Google im Kontext des Site Reliability Engineering (SRE) beschriebenen *Four Golden Signals*: Latenz, Traffic, Fehler und Sättigung Ewaschuk, 2017. Diese Signalarten erfassen grundlegende technische Symptome eines Dienstes und eignen sich als Ausgangspunkt zur Ableitung erster Monitoring-Indikatoren. Für den produktiven Betrieb liefern sie jedoch primär Rohinformationen, die erst durch Kontext, Interpretation und Bewertung für Menschen handlungsrelevant werden. Google SRE weist selbst darauf hin, dass die Wirksamkeit solcher Signale maßgeblich davon abhängt, wie sie in Alerting, Entscheidungsprozesse und betriebliche Abläufe eingebettet sind Ewaschuk, 2017.

Die Open-Source-Plattform *frag.jetzt* verfügt über eine solide Entwicklungs- und Deployment-Infrastruktur, jedoch ist die Erfassung und Auswertung von Betriebsdaten derzeit nicht als durchgängiges Observability-Konzept ausgelegt. Metriken, Logs, Traces und Alerting werden nicht konsistent entlang des gesamten Anfragepfads betrachtet, wodurch Zusammenhänge zwischen Nutzerverhalten, Backend-Verarbeitung und externen Abhängigkeiten nur eingeschränkt nachvollziehbar sind. Diese Einschätzung stützt sich auf die öffentlich verfügbare Projektdokumentation und Architekturübersicht des frag.jetzt-Repositories TH Mittelhessen und arsnova, 2025 sowie auf allgemeine Beobachtungen zu typischen Defiziten in vergleichbaren Systemen Giamattei et al., 2024.

Ziel dieser Arbeit ist die Entwicklung einer Observability-Strategie für *frag.jetzt*, bei der technische Messgrößen systematisch mit nutzungsbezogenen Fragestellungen verknüpft werden. Die Four Golden Signals dienen dabei als unterstützender Referenzrahmen zur Strukturierung der Beobachtungsdaten, nicht jedoch als alleinige Grundlage für betriebliche Entscheidungen. Die Signale werden servicespezifisch für das PWA-Frontend, das Spring-Boot-Backend, die PostgreSQL-Datenbank sowie angebundene KI-Dienste konkretisiert und in messbare Indikatoren überführt. Ergänzend wird untersucht, welche Open-Source-Observability-Werkzeuge die Anforderungen von *frag.jetzt* im Hinblick auf Erfassung und betriebliche Nutzbarkeit am besten erfüllen.

Ein weiterer Schwerpunkt der Arbeit liegt auf der Ausgestaltung eines Alerting-Ansatzes, der Auffälligkeiten innerhalb der Plattform mit Fragen der Relevanz und Dringlichkeit verknüpft. Dabei steht nicht allein die Detektion von Abweichungen im Vordergrund, sondern insbesondere die Bewertung, ob eine Handlung erforderlich ist, wen ein Problem betrifft und welche Auswirkungen es auf die Nutzung der Plattform hat. Forschungsarbeiten zeigen, dass eine Kombination aus priorisiertem Alerting und datenbasierten Analyseverfahren dazu beitragen kann, irrelevante Alarmierungen zu reduzieren und menschliche Entscheidungsprozesse im Betrieb zu unterstützen Jalalvand et al., 2024. Aufbauend auf den definierten Alerts werden Runbooks konzipiert, die strukturierte Diagnose- und Handlungsschritte für wiederkehrende Incident-Szenarien bereitstellen.

1.1 Forschungsfragen

Die Forschungsfragen dieser Arbeit lauten:

1. FF1: Wie können die Four Golden Signals für die spezifischen Services von *frag.jetzt* (PWA-Frontend, Spring Boot Backend, PostgreSQL, KI-Services) konkret definiert und instrumentiert werden?
2. FF2: Welche Observability-Stack-Kombination (Prometheus/Grafana, ELK Stack, Jaeger) ist am besten geeignet für die Monitoring-Anforderungen von *frag.jetzt*?
3. FF3: Wie kann ein intelligentes Alert-System entwickelt werden, das False Positives minimiert und actionable insights für verschiedene Stakeholder-Rollen (DevOps, Entwickler, Product Owner) bereitstellt?

2 Theoretische Fundierung

2.1 Abgrenzung von Monitoring und Observability in verteilten Systemen

Für die Entwicklung einer Observability-Strategie ist zunächst eine begriffliche Trennung von Monitoring und Observability erforderlich. Monitoring bezeichnet die kontinuierliche Erfassung und Analyse von Betriebsdaten wie Logs und Metriken, um den aktuellen Zustand eines Systems sichtbar zu machen (Usman et al., 2022, S. 86907). Es setzt voraus, dass bekannte Zustandsindikatoren und erwartbare Abweichungen vorab definiert wurden, und prüft, ob diese Grenzwerte eingehalten werden.

Observability geht über diese hypothesengeleitete Überwachung hinaus. Im Kern beschreibt der Begriff die Fähigkeit, aus den externen Ausgaben eines Systems auf dessen interne Zustände zu schließen (Usman et al., 2022, S. 86907). Entscheidend ist dabei die Möglichkeit, auch neuartige und vorab nicht antizipierte Störungen nachvollziehen zu können. Solche unvorhergesehenen Fehlerbilder, die als *unknown unknowns* bezeichnet werden, entstehen gerade in verteilten Architekturen aus dem Zusammenspiel vieler parallel agierender Komponenten (Banerjee und Singh, 2024, S. 916). Die Interviewstudie von Niedermaier et al. (2019, S. 11) bestätigt diese Perspektive aus der Industriepraxis: Unternehmen wechseln zunehmend von einer komponentenisolierten Zustandsprüfung hin zu einer nutzungs- und geschäftsprozessorientierten Gesamtsicht.

Monitoring bildet damit die notwendige Voraussetzung für Observability, wird jedoch nicht davon ersetzt (Usman et al., 2022, S. 86907). Für die vorliegende Arbeit bedeutet das, eine Strategie, die sich auf das reine Erfassen bekannter Grenzwerte beschränkt, reicht für ein verteiltes System wie frag.jetzt nicht aus. Benötigt wird vielmehr ein Ansatz, der technische Messdaten mit diagnostischer Tiefe und Nutzerperspektive verbindet.

2.2 Besonderheiten verteilter und cloud-nativer Anwendungen

Verteilte und cloud-native Anwendungen werden häufig als Microservices-Architekturen umgesetzt, die aus zahlreichen, unabhängig deploybaren Diensten bestehen und in Containern betrieben werden (Sakinala, 2025, S. 102). Die resultierende Komplexität lässt sich auf vier Eigenschaften verdichten: Modularität, Verteilung, Dynamik und Flüchtigkeit (Usman et al., 2022, S. 86908). Frühere Überwachungsansätze, die auf einzelne Schichten zugeschnitten waren, stoßen unter diesen Bedingungen an Grenzen, weil sie Interaktionen zwischen Diensten zur Laufzeit nur unzureichend abbilden (Usman et al., 2022, S. 86908).

Der Grund liegt in der starken Laufzeitkopplung trotz struktureller Entkopplung. Ein Leistungsproblem in einem Dienst kann durch Zustandsänderungen in einer ganz anderen Komponente verursacht werden, etwa durch verzögerte Datenbankzugriffe oder überlastete Downstream-Pfade.

Isolierte Überwachungskonzepte ohne übergreifenden Kontext führen zu diagnostischen Lücken und erschweren die Fehlerbehebung erheblich (Niedermaier et al., 2019, S. 8).

Daraus ergibt sich die Notwendigkeit einer Ende-zu-Ende-Sicht. Distributed Tracing gilt als das Verfahren, mit dem Serviceaufrufketten pro Anfrage nachvollzogen werden können (Li et al., 2022, S. 4). Ein Trace erfasst den kausal zusammenhängenden Ablauf einer Anfrage durch mehrere Stationen eines verteilten Systems und macht sichtbar, wo Verzögerungen oder Fehler auftreten. Die dafür erforderliche Weitergabe von Identifikatoren entlang des Anfragepfads ist eine entscheidende Voraussetzung, um Fehlerursachen überhaupt eingrenzen zu können (Niedermaier et al., 2019, S. 11). Abbildung 2.1 verdeutlicht den Unterschied zwischen lokaler Komponentenüberwachung und einer durchgängigen Anfrageverfolgung.

Gegenüberstellung: Lokales Komponentenmonitoring vs. Durchgängige Anfrageverfolgung

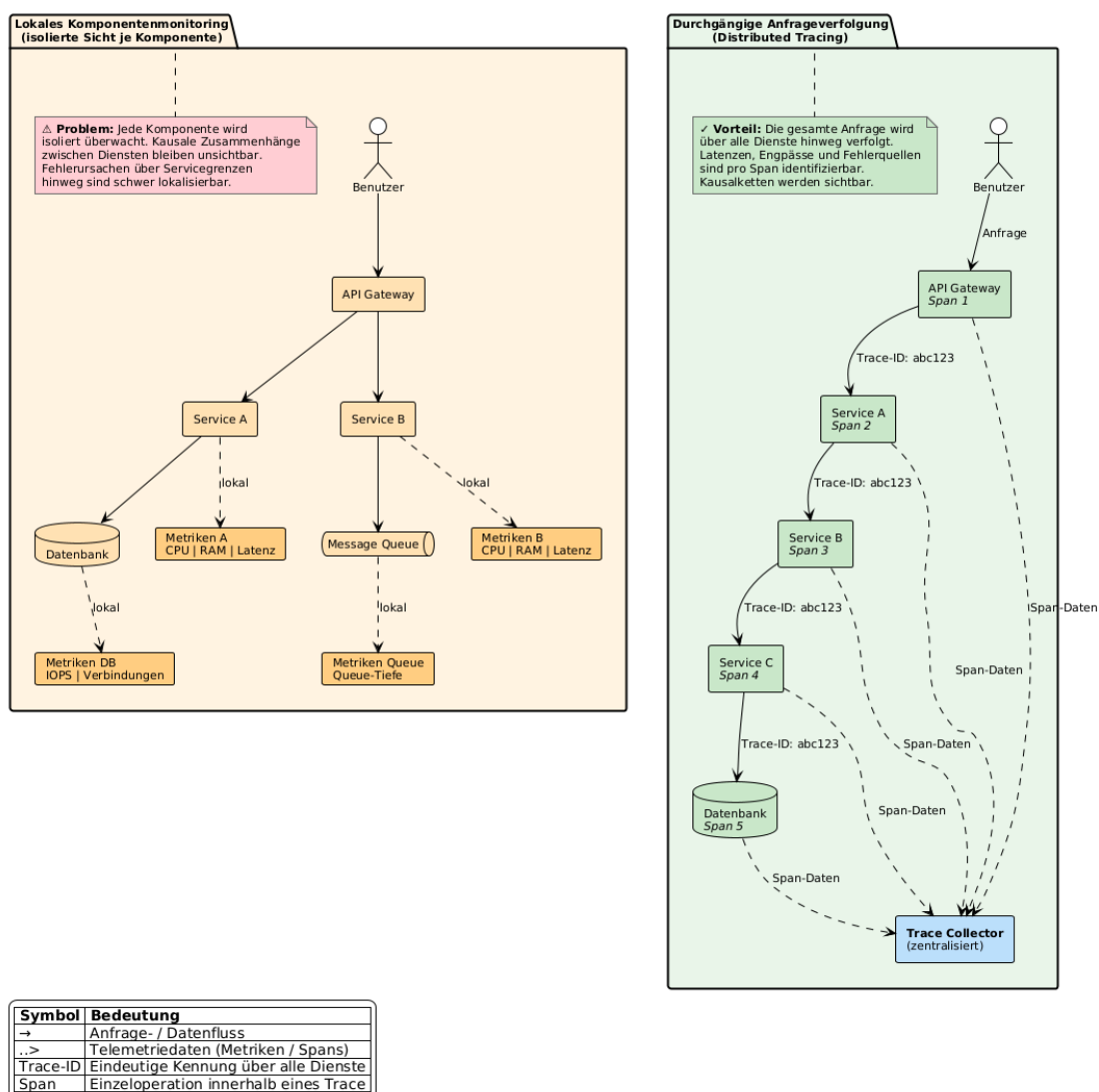


Abbildung 2.1: Gegenüberstellung von lokalem Komponentenmonitoring und durchgängiger Anfrageverfolgung über mehrere Dienste (Distributed Tracing). Quelle: Eigene Darstellung.

2.3 Telemetriearten und ihr Zusammenspiel: Logs, Metrics und Traces

Für die Beobachtbarkeit verteilter Anwendungen sind Logs, Metrics und Traces die drei am weitesten verbreiteten Telemetrieformen (Li et al., 2022, S. 6; Usman et al., 2022, S. 86912–86913; Sakinala, 2025, S. 102). Keine dieser Formen liefert für sich genommen eine vollständige Betriebssicht. Ihr Nutzen entsteht erst aus den jeweils unterschiedlichen Perspektiven, die sie auf dasselbe Laufzeitverhalten eröffnen.

Logs dokumentieren diskrete Ereignisse in zeitlicher Abfolge und eignen sich insbesondere für die Analyse unerwarteter Fehlerzustände (Albuquerque und Correia, 2025, S. 2). Allerdings erzeugen Microservices-Architekturen erhebliche Logmengen. Die zeitliche Zuordnung von Logereignissen über mehrere Dienste hinweg erfordert strukturierte Formate und Korrelationskennungen (Sakinala, 2025, S. 102). Zudem ist die Ableitung von Trends aus Logs vergleichsweise aufwendig, da die Verarbeitung großer Textmengen rechenintensiv ist (Albuquerque und Correia, 2025, S. 11).

Metrics verdichten Laufzeitverhalten zu numerischen Messwerten, die über die Zeit aggregiert werden können und sich vergleichsweise einfach speichern, abfragen und korrelieren lassen (Usman et al., 2022, S. 86913). Sie eignen sich besonders für Zustandsentwicklungen, Schwellenüberwachung und Alarmauslösung. Gleichzeitig abstrahieren Metrics stärker als Logs: Sie zeigen, dass eine Latenz steigt, erklären jedoch nicht automatisch den Grund. Reine Infrastrukturmetriken können für anwendungsbezogene Diagnosen zu grobkörnig ausfallen (Albuquerque und Correia, 2025, S. 11).

Traces erlauben die Rekonstruktion konkreter Anfragepfade über mehrere Komponenten hinweg und ergänzen Metriken um eine prozessbezogene Perspektive (Albuquerque und Correia, 2025, S. 2). Allerdings ist Tracing aufwendiger als die Erhebung einzelner Metriken, da die Instrumentierung ressourcenintensiver sein kann (Giamattei et al., 2024, S. 15) und Sampling-Strategien erforderlich werden, um den Overhead in ein vertretbares Verhältnis zum diagnostischen Nutzen zu bringen (Albuquerque und Correia, 2025, S. 10).

Der komplementäre Charakter dieser Datenarten ist in der Literatur breit anerkannt (Li et al., 2022, S. 6). Über Korrelationskennungen können Logs mit Traces verknüpft werden, sodass zeitliche Abläufe und Fehlermeldungen gemeinsam auswertbar werden (Albuquerque und Correia, 2025, S. 10). Eine leistungsfähige Observability-Plattform muss Anomalien daher nicht isoliert, sondern im Zusammenspiel aller drei Datenarten sichtbar machen (Usman et al., 2022, S. 86914).

2.4 OpenTelemetry als Standard für herstellerübergreifende Instrumentierung

Je stärker Observability auf mehrere Datenarten, Sprachen und Werkzeuge angewiesen ist, desto dringender wird eine gemeinsame Instrumentierungsgrundlage. Heterogene Monitoringlandschaften leiden häufig unter proprietären Formaten und Integrationsengpässen (Sakinala, 2025, S. 105;

Giamattei et al., 2024, S. 17). OpenTelemetry adressiert dieses Problem als herstellernerneutrale Instrumentierungsschicht, die aus der Zusammenführung von OpenCensus und OpenTracing hervorgegangen ist und die Erzeugung sowie Erfassung von Telemetriedaten vereinheitlicht (Usman et al., 2022, S. 86911). Offene Spezifikationen erlauben es Entwicklungsteams, ihren Code zu instrumentieren, ohne sich an ein bestimmtes Analyse-Backend zu binden (Li et al., 2022, S. 22; Sakinala, 2025, S. 106).

Zugleich darf OpenTelemetry nicht mit einem vollständigen Observability-Stack gleichgesetzt werden. Faseeha et al. (2025, S. 72030) unterscheiden explizit, dass OpenTelemetry die Instrumentierung übernimmt, während andere Werkzeuge für Speicherung, Darstellung und Alarmierung zuständig bleiben. Instrumentierung erzeugt stets zusätzlichen Entwicklungs- und Pflegeaufwand und muss deshalb zur Entwicklungsgeschwindigkeit des jeweiligen Projekts passen (Giamattei et al., 2024, S. 15). Für die spätere Stack-Evaluation ist diese Differenzierung entscheidend. Die Wahl der Instrumentierungsschicht und die Wahl der Analyse-Backends sind zwei getrennte Architekturentscheidungen.

2.5 Die Four Golden Signals als heuristischer Bezugsrahmen

Die Four Golden Signals (*Latency*, *Traffic*, *Errors* und *Saturation*) bilden in der SRE-nahen Literatur einen bewusst reduzierten, aber praxistauglichen Rahmen zur Beobachtung nutzerorientierter Dienste. Ewaschuk (2017, S. 7) beschreibt sie als die vier Größen, auf die sich die Überwachung eines clientseitig wahrnehmbaren Dienstes mindestens konzentrieren sollte, wenn nur ein begrenzter Satz von Metriken erhoben werden kann.

Latency erfasst die Zeit, die ein Dienst zur Bearbeitung einer Anfrage benötigt. Dabei müssen erfolgreiche und fehlgeschlagene Anfragen getrennt betrachtet werden, weil schnell ausgelieferte Fehlantworten die Gesamtstatistik verzerren können (Ewaschuk, 2017, S. 7).

Traffic bezeichnet die auf einen Dienst wirkende Nachfrage und muss als dienstspezifische Lastgröße verstanden werden, etwa als HTTP-Anfragen pro Sekunde oder als Anzahl aktiver Sitzungen (Ewaschuk, 2017, S. 7).

Errors umfassen die Rate fehlgeschlagener Anfragen. Dabei geht es nicht nur um explizite Fehler wie HTTP-500-Antworten, sondern auch indirekte Fehlzustände, etwa sachlich falsche Ergebnisse oder Antwortzeiten jenseits zugesicherter Schwellen (Ewaschuk, 2017, S. 8).

Saturation beschreibt, wie stark ein Dienst seine limitierenden Ressourcen ausschöpft. Da Dienste häufig bereits vor Erreichen ihrer nominellen Vollauslastung Leistungsabfälle zeigen, werden steigende Latenzen in der Praxis als Frühindikator für bevorstehende Kapazitätsengpässe interpretiert (Ewaschuk, 2017, S. 8).

Diese vier Größen werden auch von Usman et al. (2022, S. 86913–86914) als wesentliche Messperspektiven eingeordnet, die als Grundlage für Alerting, Fehlersuche und Kapazitätsplanung dienen. Solche Metriken sind jedoch nur dann analytisch wertvoll, wenn sie an die konkrete Rolle des jeweiligen Dienstes angepasst werden (Albuquerque und Correia, 2025, S. 12). Die

Stärke des Rahmenwerks liegt in seiner Ordnungsfunktion: Es zwingt dazu, Beobachtungsgrößen entlang weniger, aus Nutzersicht aussagekräftiger Dimensionen zu strukturieren.

2.6 SLIs, SLOs und Error Budgets

Während die Four Golden Signals einen Orientierungsrahmen für relevante Beobachtungsgrößen liefern, beantworten sie noch nicht, welches Niveau an Zuverlässigkeit als ausreichend gelten soll. Service Level Indicators (SLIs) binden konkrete Messgrößen an servicebezogene Zielgrößen (SLOs) zurück, die geschäftsrelevante Anforderungen aus der Nutzerperspektive fassbar machen (Niedermaier et al., 2019, S. 11). Damit entsteht eine Brücke zwischen technischer Messung und fachlicher Erwartung, die als gemeinsamer Referenzrahmen für die teamübergreifende Zusammenarbeit dient (Niedermaier et al., 2019, S. 12).

Gerade in verteilten Anwendungen ist diese Brücke wichtig, weil isolierte Infrastrukturmetriken wenig darüber aussagen, ob ein Dienst seine Aufgabe aus Nutzersicht erfüllt. Alarme sollten nicht auf beliebige Grenzwertverletzungen reagieren, sondern an Schwellen orientiert sein, deren Überschreitung die Einhaltung definierter SLOs gefährdet (Vinnakota und Kolla, 2025, S. 176).

Das Error Budget bezeichnet den rechnerisch aus einem SLO abgeleiteten Spielraum für tolerierte Unzuverlässigkeit innerhalb eines festgelegten Zeitfensters (Dasari, 2025, S. 993). Die praktische Bedeutung liegt in der Steuerungsfunktion: Solange Budget vorhanden ist, können Änderungen mit kontrolliertem Risiko ausgerollt werden. Ist es aufgebraucht, erhält Stabilisierung Vorrang gegenüber Weiterentwicklung (Dasari, 2025, S. 993–994). Damit verändert sich auch die Funktion von Alerting. Ein Alarm, der lediglich auf lokale Ressourcenauslastung oder starre Einzelgrenzwerte reagiert, kann technisch korrekt und dennoch betrieblich wenig hilfreich sein.

2.7 Alert Fatigue, Actionable Alerts und Runbooks

Die Literatur zu Monitoring und Alerting zeigt übereinstimmend, dass nicht die Menge an Alarmen, sondern deren geringe Handlungsqualität ein zentrales Betriebsproblem darstellt. Bei zu hoher Frequenz werden Warnungen nur noch überflogen oder ignoriert, sodass echte Störungen im Rauschen untergehen können (Ewaschuk, 2017, S. 4). Dieser Effekt wird als Alert Fatigue bezeichnet (Vinnakota und Kolla, 2025, S. 173).

In einer strukturell vergleichbaren Problemlage zeigen Jalalvand et al. (2024, S. 2–3) für Security Operations Centres, dass hohe Alarmvolumina, zahlreiche False Positives und mangelnde Priorisierung Analyseteams überlasten. Obwohl der fachliche Kontext ein anderer ist, verdeutlicht der Befund, dass Alarmmüdigkeit ein allgemeines Entwurfsproblem alarmintensiver Betriebsumgebungen ist und nicht nur ein Randproblem einzelner Werkzeuge.

Vor diesem Hintergrund gewinnt der Begriff des *actionable alert* an Bedeutung. Alerts sollten nur dann ausgelöst werden, wenn eine tatsächliche oder unmittelbar drohende Beeinträchtigung für

den Nutzer vorliegt (Ewaschuk, 2017, S. 11–12). Beim Eintreffen eines Alarms sollte der operative Kontext so aufbereitet sein, dass unmittelbar erkennbar wird, welches Problem vorliegt, welche Ursachen wahrscheinlich sind und welche Maßnahmen eingeleitet werden müssen (Vinnakota und Kolla, 2025, S. 174). Wirksame Alerting-Strategien müssen zudem zwischen handlungsrelevanten Hinweisen und bloßem Hintergrundrauschen differenzieren. Als problematisch gelten starre Schwellwerte in dynamischen Umgebungen, weil sie saisonale Schwankungen oder Abhängigkeiten zwischen Diensten unzureichend berücksichtigen (Vinnakota und Kolla, 2025, S. 173, 176).

Eng mit handlungsfähiger Alarmierung ist die Rolle von Runbooks verbunden. Jede kritische Meldung sollte mit einem Runbook verknüpft sein, das Verweise auf relevante Observability-Werkzeuge, vordefinierte Behebungsschritte, klare Eskalationspfade und Angaben zur Zuständigkeit enthält (Vinnakota und Kolla, 2025, S. 174). Für wiederkehrende, schematische Reaktionsmuster empfiehlt die SRE-Literatur eine konsequente Automatisierung, damit algorithmische Alarmreaktionen nicht dauerhaft menschliche Aufmerksamkeit binden (Ewaschuk, 2017, S. 14).

3 Systemanalyse und Anforderungsdefinition: „frag.jetzt“

Die folgende Systemanalyse basiert auf einer KI-gestützten Auswertung des Quellcodes, der Deployment-Konfigurationen und der Abhängigkeitsstruktur der fünf Repositories des frag.jetzt-Ökosystems.¹ Ziel ist die Identifikation observability-relevanter Architekturmerkmale, Risikopunkte und bestehender Instrumentierungslücken als Grundlage für die Anforderungsdefinition.

3.1 Fachlicher Kontext und Nutzungsszenario

frag.jetzt ist ein webbasiertes Audience-Response-System für Lehrveranstaltungen. Teilnehmende können anonym Fragen stellen, auf Beiträge abstimmen und an Live-Umfragen teilnehmen. Ergänzend bietet das System KI-gestützte Funktionen wie Keyword-Extraktion und thematische Zusammenfassungen. Das Nutzungsmuster ist durch synchrone Spitzenlast geprägt. Innerhalb kurzer Veranstaltungsfenster sind mehrere hundert Teilnehmende gleichzeitig aktiv, während die Last außerhalb nahe null liegt. Für die Observability-Strategie folgt daraus, dass Engpässe vor allem unter Spitzenlast erkannt werden müssen – nicht im Tagesdurchschnitt. Niedermaier et al. (2019, S. 8) identifizieren solche Dynamik als zentrale Herausforderung beim Monitoring verteilter Systeme, da konventionelle Schwellenwerte transiente Lastspitzen nicht zuverlässig abbilden.

3.2 Technische Architektur

Das Ökosystem umfasst fünf Repositories, die über Docker Compose auf einem einzelnen Host zu zehn Containern orchestriert werden. Kubernetes kommt nicht zum Einsatz. Die Anwendung gliedert sich in vier applikationstragende Dienste (Tabelle 3.1) und mehrere Infrastrukturkomponenten.

Die Infrastruktur umfasst die PostgreSQL-Hauptdatenbank, eine dedizierte Keycloak-Datenbank, eine separate AI-Datenbank mit logischer Replikation aus der Hauptdatenbank, RabbitMQ als Message Broker, Keycloak für Authentifizierung (OIDC/SSO) und einen E-Mail-Dienst. Für die Observability-Strategie ist die Deployment-Topologie wesentlich: Alle Container laufen auf einem einzelnen Host ohne konfigurierte Ressourcenlimits (CPU, Speicher) und ohne Health-Check-Direktiven auf Container-Ebene.

1 Quellcode abrufbar unter <https://gitlab.arsnova.eu/arsnova/frag.jetzt/> (abgerufen am 28. März 2026).

Tabelle 3.1: Applikationstragende Dienste und ihre Observability-Relevanz. Quelle: Eigene Darstellung.

Dienst	Technologie	Observability-Relevanz
Frontend	Angular, Nginx	Einstiegspunkt; Nginx routet an Backend (/api), WS-Gateway (/gateway), Keycloak (/auth), AI Provider (/ai)
Backend	Spring Boot, WebFlux, R2DBC	Zentraler kritischer Dienst: 95 Endpunkte, 19 Controller
WS-Gateway	Kotlin, Netty, STOMP	Echtzeit-Updates via RabbitMQ; Session-Tracking in-memory
AI Provider	Python, FastAPI, Lang-Chain	19 LLM-Provider; externe Abhängigkeiten ohne Timeout

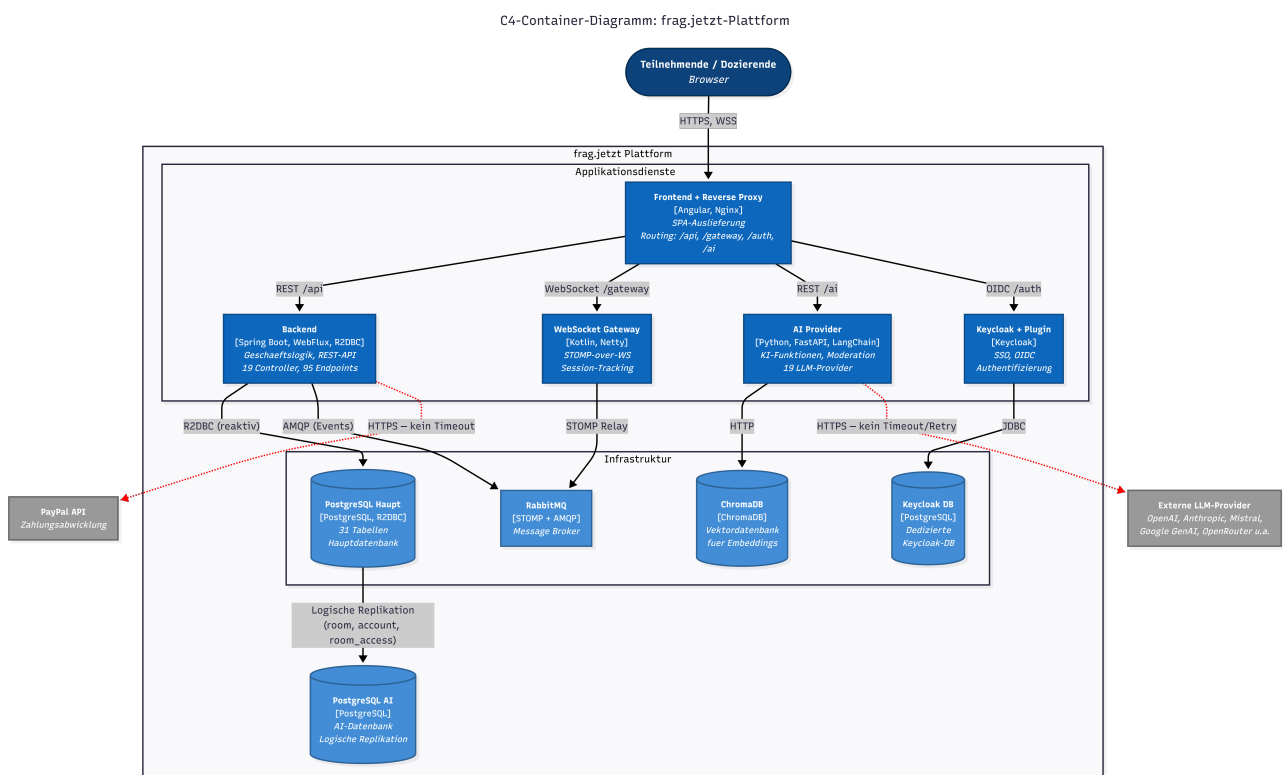


Abbildung 3.1: C4-Container-Diagramm der frag.jetzt-Architektur mit Applikationsdiensten, Infrastrukturkomponenten und Kommunikationswegen (Vollseitenansicht des Diagramms in Anhang A.2). Quelle: Eigene Darstellung.

3.3 Kritische User Journeys

Eine wirksame Observability-Strategie orientiert sich an den tatsächlichen Nutzungspfaden, nicht an abstrakten Systemkomponenten (Niedermaier et al., 2019, S. 10–11). Für frag.jetzt lassen sich drei geschäftskritische Abläufe identifizieren.

3.3.1 Journey 1: Frage stellen.

Die Frage wird vom Frontend per REST an das Backend übermittelt, dort validiert, in der Datenbank persistiert – wobei ein Trigger eine Kommentarnummer generiert – und über RabbitMQ als Echtzeit-Update an alle Raum-Clients weitergeleitet. Der Ablauf durchläuft vier Komponenten; Messgrößen sind Backend-Antwortzeit, Dauer des DB-Inserts und WebSocket-Zustelllatenz.

3.3.2 Journey 2: Live-Voting.

Votes werden per REST persistiert. Ein Trigger aktualisiert synchron die Score-Spalte der zugehörigen Frage; bei parallelen Abstimmungen entsteht Row-Level Contention, die unter Spitzenlast zu steigenden Antwortzeiten führen kann (vgl. Abschnitt 3.4, R2).

3.3.3 Journey 3: KI-Zusammenfassung.

Die Anfrage gelangt vom Frontend über das Backend an den AI Provider, der einen externen LLM-Dienst aufruft. Kein externer Aufruf im System hat einen konfigurierten Timeout; ein nicht reagierender Upstream-Dienst kann daher Request-Laufzeiten unbegrenzt verlängern und die rechtzeitige Antwortlieferung an das Frontend gefährden.

3.4 Risikopunkte und Zuordnung zu den Golden Signals

Aus der Architekturanalyse lassen sich sechs Risikopunkte ableiten, die in Tabelle 3.2 den betroffenen Golden Signals und ihrer Nutzerwirkung zugeordnet werden.

3.4.1 R1: Fehlende Timeouts und Resilienz.

Weder Backend noch AI Provider konfigurieren Timeouts, Retries oder Circuit Breaker auf ausgehenden HTTP-Aufrufen. Ein nicht reagierender externer Dienst kann Request-Laufzeiten unbegrenzt verlängern und die Ressourcenauslastung erhöhen.

3.4.2 R2: Datenbankengpässe bei Massenabstimmungen.

Die Vote-Trigger führen bei jedem Schreibvorgang einen zusätzlichen `SELECT` und `UPDATE` auf der Comment-Tabelle aus; bei gleichzeitigen Abstimmungen entsteht Row-Level Contention.

3.4.3 R3: Backend als Single Point of Failure ohne Observability.

Nahezu alle Nutzerinteraktionen laufen über das Backend, das gleichzeitig über keinerlei Observability-Infrastruktur verfügt – weder Actuator noch Metriken, weder Tracing noch strukturierte Logs. Fehler bleiben unsichtbar, bis Endnutzende Symptome wahrnehmen.

3.4.4 R4: WebSocket-Verbindungsverlust.

Für RabbitMQ existieren keine Health-Checks. Ein Broker-Ausfall unterbricht alle Echtzeit-Updates, bleibt aber ohne Überwachung der Queue-Tiefe und Subscription-Anzahl zunächst unbemerkt.

3.4.5 R5: Fehlende Ressourcenlimits.

Kein Container definiert CPU- oder Speicherlimits. Ein einzelner Dienst mit unkontrolliertem Ressourcenverbrauch kann durch Ressourcenverdrängung alle übrigen Dienste beeinträchtigen.

3.4.6 R6: Single-Host-Topologie als systemweiter Ausfallpfad.

Alle zehn Container laufen auf einem gemeinsamen Host. Ein Hardware- oder Betriebssystemausfall betrifft das Gesamtsystem; in Kombination mit R5 entsteht ein einzelner Sättigungspfad, über den ein Dienst alle übrigen destabilisieren kann.

Tabelle 3.2: Zuordnung der Risikopunkte zu Golden Signals und Nutzerwirkung. Quelle: Eigene Darstellung.

Risiko	Dienst(e)	Signale	Nutzerwirkung
R1	Backend, AI Provider	Latency, Errors	Hängende Anfragen, Timeouts
R2	Backend, PostgreSQL	Latency, Saturation	Verzögerte Votes in Spitzenlast
R3	Backend	Alle	Fehler erst durch Nutzerbeschwerden sichtbar
R4	WS-Gateway, RabbitMQ	Errors, Traffic	Ausbleibende Echtzeit-Updates
R5	Alle Container	Saturation	Systemweite Instabilität
R6	Gesamtsystem	Saturation, Errors	Totalausfall bei Host-Ausfall

3.5 Observability-Ist-Zustand

Die Quellcode-Analyse ergibt, dass die bestehende Observability-Infrastruktur minimal ist. Das Backend besitzt weder Actuator noch Metriken, kein Tracing und keine strukturierten Logs. Das WS-Gateway konfiguriert einen Health- und Prometheus-Endpunkt, jedoch ist die benötigte Micrometer-Registry nicht als Abhängigkeit deklariert. RabbitMQ hat das Prometheus-

Plugin aktiviert, der Port wird aber nicht exponiert. Systemweit fehlen Distributed Tracing, Korrelations-IDs, zentrale Log-Aggregation und ein Monitoring-Stack vollständig. Störungen werden gegenwärtig erst wahrgenommen, wenn Endnutzende direkt betroffen sind – ein Zustand, den Niedermaier et al. (2019, S. 9–10) als Folge einer rein reaktiven Monitoring-Implementierung einordnen.

3.6 Ableitung der Observability-Anforderungen

Aus Architektur, Kernabläufen, Risikopunkten und Ist-Zustand lassen sich fünf Anforderungen ableiten.

3.6.1 A1: Ende-zu-Ende-Nachvollziehbarkeit über Dienstgrenzen.

Die User Journeys durchlaufen jeweils mehrere Dienste; kausale Zusammenhänge zwischen Störungen sind ohne dienstübergreifende Sichtbarkeit nicht erkennbar. Albuquerque und Correia (2025, S. 6) beschreiben *Distributed Tracing* als grundlegendes Entwurfsmuster für diese Anforderung: Jeder eingehenden Anfrage wird eine eindeutige ID zugewiesen, die über alle Dienste propagiert und in Logs sowie Spans mitgeführt wird, sodass der gesamte Ausführungspfad rekonstruierbar ist.

3.6.2 A2: Mehrstufige Messbarkeit entlang der Golden Signals.

Die Instrumentierung muss auf drei Ebenen erfolgen: (1) Browser-Telemetrie im Frontend für Nutzererfahrungsmetriken, (2) Service-Metriken für Latenz, Durchsatz, Fehlerrate und Sättigung der vier Applikationsdienste und der Datenbank, sowie (3) Infrastruktur-Metriken für Container-Ressourcen, Verbindungspools und Queue-Tiefen. Albuquerque und Correia (2025, S. 10, 16) beschreiben *Application Metrics* und *Infrastructure Metrics* als komplementäre Entwurfsmuster, deren Zusammenspiel erst eine ganzheitliche Systemsicht ermöglicht (Albuquerque & Correia, 2025, S. 22).

3.6.3 A3: Strukturiertes, korrelierbares Logging.

Das Logging muss auf strukturierte Formate (JSON) mit Korrelations-IDs umgestellt und zentral aggregiert werden, damit Logeinträge verschiedener Dienste einer gemeinsamen Anfrage zugeordnet werden können (Albuquerque & Correia, 2025, S. 9–10).

3.6.4 A4: Handlungsfähige Alarmierung.

Alarme sollen auf nutzerwirksame Zustände reagieren – steigende Antwortzeiten, wachsende Fehlerraten oder nicht erreichbare Schnittstellen – und ausreichend Kontext für eine gezielte Erstreaktion enthalten. Formale SLOs existieren derzeit nicht und können als zukünftige Weiterentwicklung betrachtet werden.

3.6.5 A5: Beobachtbarkeit der Infrastrukturschicht.

Die Single-Host-Topologie (R6) und die fehlenden Ressourcenlimits (R5) erzeugen blinde Flecken, die nur durch Infrastruktur-Metriken sichtbar werden. Die Strategie muss Container-Ressourcenverbrauch, Datenbankverbindungspool-Auslastung, RabbitMQ-Queue-Tiefen und Dienstverfügbarkeit über Health-Endpunkte einbeziehen. Niedermaier et al. (2019, S. 10) formulieren eine solche ganzheitliche Sicht über System- und Infrastrukturebenen hinweg als Kernanforderung an Monitoring-Konzepte verteilter Systeme.

Diese Anforderungen bilden die Grundlage der folgenden Kapitel: A1–A3 adressieren primär Forschungsfrage 1, A2 und A5 liefern Evaluationskriterien für Forschungsfrage 2, und A4 bildet die Grundlage für Forschungsfrage 3.

4 Methodisches Vorgehen und Bewertungsrahmen

Dieses Kapitel legt das methodische Vorgehen offen, mit dem aus den theoretischen Grundlagen (Kapitel 2) und der Systemanalyse (Kapitel 3) eine konkrete Observability-Strategie für frag.jetzt entsteht.

4.1 Arbeitslogik der Arbeit

Die Arbeit folgt einer konstruktiv-artefaktorientierten Arbeitslogik. Auf Basis der theoretischen Fundierung und der Systemanalyse werden Observability-Anforderungen abgeleitet, darauf aufbauend eine Strategie entwickelt, in ausgewählten Komponenten prototypisch umgesetzt und anhand zuvor definierter Kriterien bewertet.

Die Arbeitsschritte gliedern sich in fünf Phasen: Die theoretische Fundierung (Kapitel 2) erarbeitet die begrifflichen Grundlagen. Die Systemanalyse (Kapitel 3) überführt diese in den Kontext von frag.jetzt und leitet fünf Observability-Anforderungen (A1–A5) ab. Die Stack-Evaluation (Kapitel 5) bewertet verfügbare Technologiekombinationen. Die Strategiekonzeption mit prototypischer Umsetzung (Kapitel 6 und 7) bildet den Kern der Eigenleistung. Abschließend prüft die Evaluation (Kapitel 8) die Strategie gegen die Anforderungen aus Kapitel 3.

Die Architekturhebung in Kapitel 3 stützt sich ergänzend auf eine KI-gestützte Quellcode-exploration mithilfe von GitHub Copilot, das als Explorationshilfe für Architekturübersichten, Controller-Strukturen und bestehende Konfigurationen diene. Sämtliche architekturrelevanten Befunde wurden anschließend durch manuelle Prüfung gegen Quellcode, Docker-Compose-Dateien und Konfigurationsartefakte validiert. Das Verfahren erfasst implizite Laufzeitabhängigkeiten außerhalb des versionierten Quellcodes nur eingeschränkt.

4.2 Vorgehen bei der Ableitung der Golden Signals pro Service

Die Herausforderung liegt weniger in der Kenntnis der vier Golden Signals als in ihrer konkreten Übersetzung auf heterogene Dienste mit unterschiedlichen Rollen und Kommunikationsmustern. Damit diese Übersetzung nachvollziehbar erfolgt, folgt die Arbeit einer fünfstufigen Ableitungslogik.

Im ersten Schritt wird die *Rolle des Dienstes im Gesamtsystem* bestimmt, weil die funktionale Einordnung maßgeblich beeinflusst, welche Beobachtungsgrößen aussagekräftig sind. Der zweite Schritt identifiziert die *kritischen Interaktionen* anhand der User Journeys und Risikopunkte aus Kapitel 3, da Metriken an realen Nutzungspfaden ausgerichtet sein sollten (Niedermaier et al., 2019, S. 10–11). Im dritten Schritt werden *geeignete Messgrößen* abgeleitet, wobei pro Golden Signal mindestens eine primäre Messgröße bestimmt wird. Nicht jedes Signal lässt sich

bei jedem Dienst als White-Box-Metrik erfassen. Bei externen KI-Services etwa ist Saturation nur indirekt beobachtbar.

Der vierte Schritt begründet *initiale Schwellenwerte*. Da für frag.jetzt keine historischen Produktivdaten vorliegen, werden operative Startschwellen auf Basis von Lasttests und Erfahrungswerten festgelegt, die in einer Staging-Umgebung überprüft und im Betrieb angepasst werden. Formale SLOs existieren derzeit nicht und werden als zukünftige Weiterentwicklung betrachtet. Niedermaier et al. (2019, S. 9) bestätigen, dass unklare nichtfunktionale Anforderungen und reaktive Monitoring-Entwicklung zu den häufigsten Herausforderungen verteilter Systeme gehören. Im fünften Schritt wird bestimmt, welche *Visualisierung und Alarmierung* aus den Messgrößen folgen. Nicht jede Metrik rechtfertigt einen Alarm, und die Zielgruppe bestimmt die Aufbereitung (Vinnakota & Kolla, 2025, S. 175).

4.3 Kriterien für Stack-Auswahl und spätere Evaluation

Die Arbeit verwendet zwei bewusst getrennte Kriterienebenen. Die Stack-Auswahl (Kapitel 5) beantwortet die Werkzeugfrage, welche Technologiekombination die Anforderungen am besten adressiert. Die Evaluation (Kapitel 8) beantwortet die Ergebnisfrage, ob die entwickelte Strategie die identifizierten Probleme tatsächlich löst.

4.3.1 Vergleichskriterien für die Stack-Auswahl

Aufbauend auf den Anforderungen A1–A5 und der einschlägigen Literatur (Sakinala, 2025, S. 104); (Faseeha et al., 2025, S. 72015); (Giamattei et al., 2024, S. 3) werden folgende acht Vergleichskriterien verwendet:

- **Abdeckung der Telemetriepfeiler.** Unterstützung von Metriken, Logs und Traces in integrierter Form.
- **Korrelationsfähigkeit.** Verknüpfung verschiedener Telemetriearten über gemeinsame Bezeichner (Trace-IDs, Korrelations-IDs).
- **OpenTelemetry-Kompatibilität.** Unterstützung des herstellerneutralen CNCF-Standards für Instrumentierung und Datenexport.
- **Dashboarding und Alerting.** Eignung für rollenspezifische Dashboards und differenzierte Alarmregeln.
- **Integrationsaufwand.** Technischer Aufwand für die Einbindung in die bestehende Docker-Compose-Topologie.
- **Ressourcenbedarf.** Zusätzlicher Speicher-, CPU- und Festplattenbedarf auf einem einzelnen Host (vgl. R6).
- **Erweiterbarkeit.** Anpassungsfähigkeit bei wachsendem Telemetriedatenvolumen oder zusätzlichen Diensten.
- **Community und Ökosystem.** Breite der Nutzerbasis, Dokumentationsreife und Framework-Integrationen.

4.3.2 Bewertungskriterien für die Gesamtstrategie

Die Gesamtstrategie wird direkt an den Anforderungen A1–A5 gemessen. Für jede Anforderung wird eine konkrete Prüffrage formuliert: Lässt sich ein Request Ende-zu-Ende rekonstruieren (A1)? Deckt die Instrumentierung alle drei Messebenen ab (A2)? Können Logeinträge über Korrelations-IDs verknüpft werden (A3)? Reagieren Alarme auf nutzerwirksame Zustände und enthalten sie ausreichend Kontext für eine Erstreaktion (A4)? Werden Container-Ressourcen, Datenbankpool-Auslastung und Queue-Tiefen erfasst (A5)?

Die Beantwortung stützt sich auf drei Evidenzquellen: die Artefaktanalyse von Instrumentierungskonfigurationen, Dashboard-Definitionen und Alarmregeln, die Konfigurations- und Instrumentierungsprüfung sowie ausgewählte Last- und Störungsszenarien in einer produktionsnahen Testumgebung. Ergänzend fließen drei übergreifende Bewertungsdimensionen ein: Stakeholder-Tauglichkeit, Abdeckung der Risikopunkte R1–R6 und Umsetzungsrealismus.

5 Evaluation und Auswahl des Observability-Stacks

Das vorliegende Kapitel wendet die in Abschnitt 4.3 hergeleiteten acht Vergleichskriterien auf drei vollständige Observability-Architekturen an, um Forschungsfrage 2 zu beantworten: Welche Observability-Stack-Kombination eignet sich am besten für die Überwachungsanforderungen von frag.jetzt?

5.1 OpenTelemetry als verbindliche Instrumentierungsschicht

Bevor die Analyse-Backends verglichen werden, wird OpenTelemetry als verbindliche Instrumentierungsgrundlage festgelegt. Die Begründung wurde in Kapitel 2 vorbereitet: OpenTelemetry vereinheitlicht die Erzeugung und Erfassung von Logs, Metriken und Traces als herstellerneutraler CNCF-Standard, ohne selbst Speicherung oder Darstellung zu übernehmen (Faseeha et al., 2025, S. 72030). Eine standardisierte Instrumentierung entkoppelt den Applikationscode von der Backend-Wahl, sodass die Analyse-Ebene austauschbar bleibt (Sakinala, 2025, S. 107). Für frag.jetzt als polyglotte Anwendung mit vier Laufzeitumgebungen (Java, Kotlin, Python, TypeScript) ist ein sprachübergreifender Instrumentierungsansatz besonders wertvoll, weil ohne einen gemeinsamen Rahmen für jede Sprache separate Bibliotheken gepflegt werden müssten (Sakinala, 2025, S. 106). Die Trägerschaft durch die CNCF sichert langfristige Weiterentwicklung (Sakinala, 2025, S. 107). Der folgende Vergleich setzt OpenTelemetry daher als gegebene Instrumentierungsschicht voraus und bewertet die Kandidaten-Stacks ausschließlich als Empfangs-, Speicher-, Visualisierung- und Alarmierungsebene.

5.2 Vergleich der Zielarchitekturen

Für den Vergleich werden ausschließlich Architekturen herangezogen, die Logs, Metriken und Traces vollständig adressieren und über eine gemeinsame Auswertungsebene verfügen. Alle drei Kandidaten basieren auf quelloffenen Kernkomponenten und werden in der Fachliteratur zur Microservices-Beobachtbarkeit regelmäßig herangezogen (Faseeha et al., 2025, S. 72019–72021).

5.2.1 Elastic Observability

Elastic Observability erweitert den klassischen ELK-Stack um APM und OTel-Integration. Der APM-Server empfängt Traces, Metriken und Logs über OTLP nativ (Elastic, n. d.). Die Stärke liegt in der tiefgreifenden Loganalyse durch Volltextindizierung (Banerjee & Singh, 2024, S. 920). Für frag.jetzt ergeben sich allerdings gewichtige Nachteile: Die vollständige Indizierung verursacht erheblichen Speicher- und Rechenaufwand (Banerjee & Singh, 2024, S. 918), und auf der Single-Host-Topologie konkurriert ein Elasticsearch-Knoten mit seinem JVM-Heap unmittelbar mit den Anwendungscontainern. Hinzu kommt die mehrfache Lizenzänderung

(Apache 2.0 → ELv2/SSPL → teils AGPL seit 2024), die die langfristige Planungssicherheit im Vergleich zu CNCF-getragenen Projekten einschränkt (Elastic, 2024).

5.2.2 Grafana-Stack: Prometheus + Loki + Tempo + Grafana

Die zweite Zielarchitektur kombiniert Prometheus für Metriken, Loki für Logs und Tempo für Traces mit Grafana als gemeinsamer Visualisierungs- und Alarmierungsplattform (Gudimetla, 2025, S. 501). Loki verfolgt einen ressourcenschonenden Ansatz, indem es ausschließlich Labels indiziert statt den vollständigen Loginhalt, was den Speicherbedarf um 50–80 % gegenüber Volltextindizierung reduziert (Faseeha et al., 2025, S. 72019–72020); (Gudimetla, 2025, S. 502). Tempo empfängt Trace-Daten über OTLP und ermöglicht deren Verknüpfung mit Logs und Metriken innerhalb von Grafana (Grafana Labs, n. d. a). Prometheus übernimmt Metrikerfassung und Kurzzeitspeicherung. Grafana unterstützt alle drei Datenquellen nativ und vereint sie in einer navigierbaren Oberfläche (Sundström, 2025, S. 16). Mimir bleibt als skalierbares Langzeit-Backend eine evolutive Ausbauoption (Gudimetla, 2025, S. 502).

5.2.3 Prometheus + Jaeger + Loki + Grafana

Die dritte Architektur verfolgt einen Bausteinansatz aus eigenständigen Open-Source-Werkzeugen. Prometheus und Jaeger sind graduated CNCF-Projekte mit aktiver Community (Giamattei et al., 2024, S. 15). Dem Vorteil der Komponentenreife steht eine erhöhte Integrationskomplexität gegenüber: Die drei Werkzeuge folgen unterschiedlichen Konfigurationsformaten und Storage-Modellen (Gudimetla, 2025, S. 501). Die Korrelation zwischen den Telemetriearten ist über Grafana grundsätzlich möglich, bleibt aber weniger eng integriert als in einer Architektur, deren Komponenten von vornherein aufeinander abgestimmt sind (Sundström, 2025, S. 17).

5.3 Vergleich und Synthese

Tabelle 5.1 fasst die Bewertung der drei Zielarchitekturen entlang der acht Vergleichskriterien zusammen. Die Einstufungen beruhen auf der qualitativen Analyse in den vorherigen Abschnitten und stellen eine argumentativ begründete Einschätzung des Autors dar, keine empirisch erhobenen Messwerte.

Elastic Observability erzielt die niedrigste Gesamtbewertung, weil der hohe Ressourcenbedarf und der komplexe Integrationspfad auf einer Single-Host-Topologie besonders ins Gewicht fallen. Die beiden Grafana-basierten Architekturen liegen eng beieinander. Der Unterschied manifestiert sich in den Kriterien Korrelation und Integrationsaufwand.

Tabelle 5.1: Vergleichende Bewertung der drei Observability-Zielarchitekturen (3 = sehr gut, 2 = befriedigend, 1 = eingeschränkt). Alle drei Kandidaten adressieren Logs, Metriken und Traces. OpenTelemetry wird als gemeinsame Instrumentierungsschicht vorausgesetzt. Quelle: Eigene Darstellung.

Kriterium		Elastic Observability (Elastic APM, ES, Kibana)		Grafana-Stack (Prom., Loki, Tempo, Grafana)		Prom./Jaeger/Loki (Prom., Jaeger, Loki, Grafana)
Telemetrie- pfeiler	3	Logs, Metriken und Traces über APM/OTel abgedeckt	3	Alle drei Pfeiler durch Prometheus, Loki und Tempo	3	Alle drei Pfeiler durch Einzelkomponenten
Korrelation	2	Trace-Log-Korrelation in Kibana; Metrik-Anbindung über Lens	3	Trace-Log-Metrik-Navigation nativ in Grafana	2	Via Grafana-Plugins, weniger eng integriert
OTel- Kompa- tibilität	3	OTLP-Empfang nativ, EDOT-Distributionen verfügbar	3	Alle Komponenten OTel-nativ	3	Alle Komponenten OTel-nativ
Dashboard. & Alerting	2	Kibana + Watcher/Rules; weniger flexibel als Grafana	3	Grafana Unified Alerting + Alertmanager	3	Grafana + Alertmanager
Integrations- aufwand	1	Logstash/Beats-Pipeline, APM-Server, JVM-Tuning	2	Mehr Container, aber homogene Konfiguration via OTel-Collector	1	Drei Konfig.-Formate und Storage-Backends
Ressourcen- bedarf	1	Elasticsearch speicherintensiv, JVM-Heap konkurriert mit Anwendung	2	Höher als Prom. allein, deutlich geringer als Elastic	2	Jaeger benötigt separates Speicher-Backend
Erweiter- barkeit	2	Nativ skalierbar, aber Cluster-Management komplex	3	Prometheus via Mimir erweiterbar; Loki/Tempo modular	3	Jede Komponente einzeln skalierbar
Community & Ökosys- tem	2	Große Community, aber Lizenzänderungen (ELv2/SSPL, seit 2024 teils AGPL)	2	Grafana Labs getragen, wachsend, Tempo jünger als Jaeger	3	Prom. u. Jaeger graduated CNCF, Loki etabliert
Summe		16		21		20

5.4 Begründete Zielentscheidung für frag.jetzt

5.4.1 Entscheidungsbegründung im Kontext von frag.jetzt

Drei Argumente sprechen für den Grafana-Stack gegenüber der Alternative aus Prometheus, Jaeger und Loki.

Das erste Argument betrifft die *Integrationskomplexität*. frag.jetzt wird von einem sehr kleinen Entwicklungsteam betrieben, das Observability erstmals einführt. Niedermaier et al. (2019, S. 9) identifizieren knappe Ressourcen und fehlende Erfahrung als eigenständige Herausforderung (C7). Die Jaeger-Variante erfordert die Pflege dreier unterschiedlicher Konfigurationsformate und Storage-Backends (Gudimetla, 2025, S. 501), während Loki und Tempo aus demselben

Ökosystem stammen und vergleichbare Konfigurationsmuster verwenden.

Das zweite Argument adressiert die *Korrelierbarkeit*. Sundström (2025, S. 16) beschreiben Grafana als die Oberfläche, die Metriken, Traces und Logs in einer navigierbaren Gesamtsicht vereint. In der alternativen Kombination wäre Jaeger als separates Trace-Backend einbindbar, die Verknüpfung bliebe jedoch weniger eng integriert, und die Jaeger-eigene Oberfläche würde als paralleles Werkzeug bestehen bleiben.

Das dritte Argument betrifft die *Austauschbarkeit bei stabilem Instrumentierungskern*. Sollte sich Tempo im Betrieb als unzureichend erweisen, ließe sich der Trace-Export im OTel Collector auf Jaeger umleiten, ohne Applikationscode anzupassen. Ebenso kann die Metrikspeicherung bei wachsendem Datenvolumen auf Mimir umgestellt werden (Gudimetla, 2025, S. 502). Diese Flexibilität ist besonders relevant, weil sich nichtfunktionale Anforderungen häufig erst im laufenden Betrieb konkretisieren (Niedermaier et al., 2019, S. 9–10).

5.4.2 Kompromisse und Einschränkungen

Die Entscheidung geht mit bewussten Kompromissen einher. Tempo ist ein jüngeres Projekt als Jaeger, das als graduated CNCF-Projekt über einen breiteren Reifegrad verfügt (Giamattei et al., 2024, S. 15). Die gewählte Architektur verzichtet auf Mimir als Metrik-Backend, weil die lokale Prometheus-TSDB für das moderate Telemetrevolumen von frag.jetzt ausreichend ist. Lokis labelbasierte Indexierung erfordert eine sorgfältige Label-Auswahl (Grafana Labs, n. d. b), was bei der überschaubaren Zahl von Diensten beherrschbar bleibt.

Diese Nachteile werden dadurch relativiert, dass die offizielle Grafana-Labs-Dokumentation für Standardkonfigurationen ausreichend ist, die Vorteile einer homogenen Komponentenfamilie für ein kleines Team überwiegen und OpenTelemetry als Instrumentierungsschicht einen späteren Wechsel einzelner Backend-Komponenten mit vertretbarem Aufwand ermöglicht. Damit beantwortet die gewählte Kombination aus Prometheus, Loki, Tempo und Grafana, instrumentiert über OpenTelemetry, Forschungsfrage 2.

6 Konzeption der Observability-Strategie für frag.jetzt

Die vorangegangenen Kapitel haben den theoretischen Bezugsrahmen gelegt, den Ist-Zustand von frag.jetzt analysiert und den Observability-Stack begründet ausgewählt. Das vorliegende Kapitel überführt diese Vorarbeiten in eine zusammenhängende Strategie und beantwortet, *was* beobachtet werden soll, *warum* gerade diese Größen betrieblich bedeutsam sind und *nach welchen Prinzipien* die Lösung aufgebaut wird.

6.1 Leitprinzipien der Strategie

Vier Prinzipien greifen die Anforderungen A1–A5 aus Kapitel 3 auf und übersetzen sie in Gestaltungsleitlinien für die nachfolgenden Entwurfsentscheidungen.

P1: Servicebezogene Messbarkeit. Jeder Dienst von frag.jetzt wird als eigenständige Beobachtungseinheit behandelt, für die Messgrößen entlang der vier Golden Signals abgeleitet werden. Das Vorgehen folgt der Empfehlung, Beobachtungsgrößen an den Schnittstellen der Dienste zu erheben, weil sich dort Symptome manifestieren, bevor tieferliegende Ursachen sichtbar werden (Ewaschuk, 2017, S. 7–8). Anforderung A2 wird damit unmittelbar adressiert.

P2: Ende-zu-Ende-Nachvollziehbarkeit. An jedem der in Abschnitt 3.2 identifizierten Dienstübergänge muss Trace-Kontext propagiert werden, damit kausal zusammenhängende Aktivitäten über Dienstgrenzen hinweg korrelierbar bleiben (Bissig, 2024, S. 32). Ohne diese Verknüpfung bleiben Logs und Metriken isolierte Datenpunkte (Sundström, 2025, S. 14–15). Das Prinzip greift Anforderung A1 und den Korrelationsaspekt von A3 auf.

P3: Rollenorientierte Aufbereitung. Unterschiedliche Stakeholder stellen unterschiedliche Fragen an dasselbe Laufzeitverhalten (Bissig, 2024, S. 54). Abschnitt 6.5 überführt dieses Prinzip in drei konkrete Sichtebenen.

P4: Schrittweise Erweiterbarkeit. Da frag.jetzt Observability erstmals einführt und knappe Ressourcen eine eigenständige Herausforderung darstellen (Niedermaier et al., 2019, S. 9–10), wird die Strategie als erweiterbarer Ausgangspunkt angelegt. Die standardisierte Instrumentierung über OpenTelemetry stellt sicher, dass Backend-Komponenten austauschbar bleiben (Gudimetla, 2025, S. 502).

6.2 Logische Zielarchitektur der Observability

Die Observability-Architektur folgt einem vierstufigen Schichtenmodell, wie es Kosińska et al. (2023, S. 73037–73038) als grundlegendes Muster für cloud-native Anwendungen beschreiben. Abbildung 6.1 zeigt den Datenfluss von der Instrumentierung über Sammlung und Speicherung bis zur Auswertung. Die werkzeugspezifische Konfiguration wird in Kapitel 7 dokumentiert.

6.2.1 Instrumentierungsschicht

Aus der Architektur von frag.jetzt ergibt sich eine differenzierte Instrumentierungsstrategie. Backend, WebSocket Gateway und AI Provider sind als Eigenentwicklungen über Auto-Instrumentierung mit selektiver manueller Ergänzung beobachtbar. Die Quellcodeanalyse in Kapitel 3 hat gezeigt, dass keine dieser Komponenten bislang Telemetrie emittiert (R3). Demgegenüber agieren die externen LLM-Provider als Black-Box-Abhängigkeiten, bei denen mindestens die Ein- und Ausgangsseite jedes Aufrufs instrumentiert werden muss (Bissig, 2024, S. 31).

OpenTelemetry übernimmt die Rolle der standardisierten Instrumentierungsschicht (vgl. Abschnitt 5.2). Die relevanten Übergabepunkte für die Kontextpropagation sind die Verbindungen zwischen Frontend und Reverse-Proxy, zwischen Reverse-Proxy und Backend, zwischen Backend und Datenbank sowie zwischen Backend und AI Provider. Eine besondere Herausforderung stellt die Echtzeitkommunikation über den Message Broker dar, da dort eine manuelle Injektion des Trace-Kontexts in das Nachrichtenformat erforderlich wird (Sundström, 2025, S. 25). Ergänzend werden Container-Metriken und datenbankspezifische Indikatoren über spezialisierte Exporter erhoben, die Prometheus-kompatible Endpunkte bereitstellen.

6.2.2 Sammel- und Verarbeitungsschicht

Der OpenTelemetry Collector fungiert als zentrale Routing-Komponente, die Instrumentierung und Backend entkoppelt (Sakinala, 2025, S. 107); (Gudimetla, 2025, S. 502). Für frag.jetzt wird der Collector als pushbasierte Empfangskomponente für anwendungsseitige Traces und Metriken konfiguriert, weil ein Push-Modell bei einer zukünftigen Erweiterung keine Umstellung erfordert (Bissig, 2024, S. 39). Infrastrukturmetriken folgen dagegen dem etablierten Pull-Modell über Prometheus-Scraping. Bei dem für frag.jetzt erwarteten geringen Datenvolumen kann zunächst auf Sampling verzichtet werden. Eine Anpassung ist erst dann vorgesehen, wenn das Telemetrevolumen die Speicherkapazität des Hosts belastet.

6.2.3 Speicherschicht

Die aufbereiteten Signale werden in drei spezialisierten Backends abgelegt: Metriken in der lokalen Prometheus-TSDB, Logs in Loki mit labelbasierter Indexierung und Traces in Tempo. Entscheidend ist die Korrelierbarkeit über die drei Speicher hinweg, die bei getrennter Speicherung nicht automatisch entsteht (Bissig, 2024, S. 44). Drei Mechanismen bilden die diagnostischen Brücken: die Trace-ID als gemeinsamer Identifikator über alle drei Pfeiler, die Injektion der Trace-ID in den Mapped Diagnostic Context (MDC) des Logging-Frameworks (Sundström, 2025, S. 20) und Metriken-Exemplars, die den direkten Übergang von einem aggregierten Metrikwert zum zugehörigen Trace ermöglichen (Bissig, 2024, S. 25); (Albuquerque & Correia, 2025, S. 12). Die Speichertrennung ist nur dann diagnostisch tragfähig, wenn diese Verknüpfungspfade von Beginn an in der Instrumentierung verankert werden.

6.2.4 Analyse- und Visualisierungsschicht

Grafana bildet die einheitliche Oberfläche für Abfrage, Visualisierung und Alarmierung aller gespeicherten Telemetriedaten (Sundström, 2025, S. 16). Die Korrelation wird über Datenquellen-übergreifende Navigation hergestellt: Von einem auffälligen Metrikpunkt lässt sich über ein Exemplar direkt zum zugehörigen Trace springen, und aus einem Trace heraus führen verknüpfte Logabfragen zum relevanten Kontext.

6.2.5 Einordnung im Kontext von frag.jetzt

Die beschriebene Vier-Schichten-Architektur orientiert sich an Referenzmodellen für Kubernetes-basierte Umgebungen (Gudimetla, 2025, S. 501). frag.jetzt weicht davon in drei Punkten ab. Erstens läuft das Ökosystem auf einem einzelnen Host, sodass der Collector nicht als Sidecar, sondern als eigenständiger Container betrieben wird. Zweitens fehlt ein Service Mesh, weshalb die Telemetrieerzeugung vollständig auf Anwendungsebene erfolgen muss. Drittens erlaubt das geringe Telemetrevolumen einen Verzicht auf Sampling und vorgelagerte Aggregation, erfordert aber eine bewusste Kapazitätsbeobachtung bei wachsendem Einsatz.

Die Architektur hält den Einstieg bewusst schlank und lässt Erweiterungspfade wie Mimir für Langzeitmetriken oder Tail-Based Sampling offen, ohne die Grundstruktur zu verändern (Borges et al., 2024, S. 1–2).

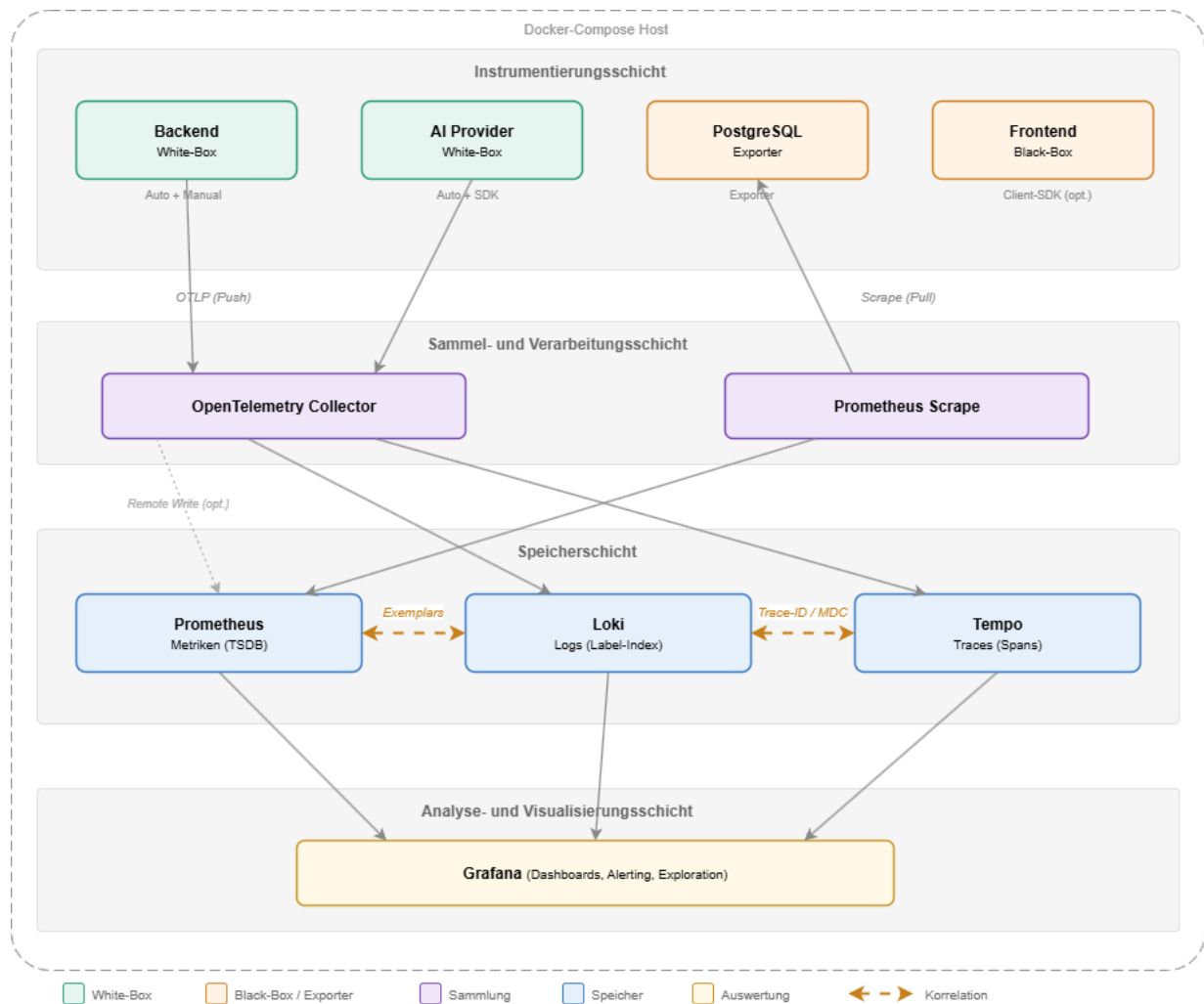


Abbildung 6.1: Logische Zielarchitektur der Observability-Pipeline fuer frag.jetzt. Das Schichtenmodell zeigt den Datenfluss von der Instrumentierung ueber Sammlung und Speicherung bis zur Auswertung. Gestrichelte Querverbindungen kennzeichnen Korrelationsmechanismen zwischen den Speicher-Backends. Quelle: Eigene Darstellung.

6.3 Servicebezogene Operationalisierung der Four Golden Signals

Die Ableitung der Beobachtungsgrößen folgt dem in Abschnitt 4.2 beschriebenen Vorgehen und konzentriert sich auf die vier für die Forschungsfragen zentralen Beobachtungseinheiten: Frontend, Backend, Datenbank und externe KI-Dienste. Nginx, RabbitMQ und das WebSocket Gateway werden als Infrastruktur- bzw. Integrationskomponenten über die Infrastrukturmetriken mitbeobachtet, erhalten aber keine eigenständige Signal-Matrix. Ewaschuk (2017, S. 7) bezeichnet die Wahl des Traffic-Indikators ausdrücklich als *high-level system-specific metric*, was bedeutet, dass die Übertragung der vier Dimensionen auf jeden Dienst eine eigenständige Entwurfsleistung erfordert. Tabelle 6.1 fasst die Ergebnisse dieser Ableitung in einer Signal-Matrix zusammen. Die folgenden Unterabschnitte erläutern nur die Besonderheiten, die sich nicht unmittelbar aus der Matrix erschließen.

6.3.1 PWA-Frontend

Das Angular-Frontend unterscheidet sich grundlegend von den serverseitigen Komponenten, weil die Ausführung im Browser stattfindet und die Infrastruktur nicht direkt kontrollierbar ist. Aus Nutzersicht manifestiert sich Latenz als wahrgenommene Ladezeit, zu der neben der Server-Antwortzeit auch clientseitige Faktoren wie Netzwerkqualität und Rendering beitragen. Basisindikatoren sind die Dauer von API-Aufrufen und die Ende-zu-Ende-Ladezeit beim Betreten eines Raums. Browserbasierte Metriken wie First Contentful Paint über das OpenTelemetry-Browser-SDK sind als optionale Erweiterung zu verstehen.

Die Nachfrageintensität lässt sich über aktive WebSocket-Verbindungen und HTTP-Anfragen pro Zeiteinheit messen. Fehlerindikatoren umfassen fehlgeschlagene API-Aufrufe (4xx/5xx), unbehandelte JavaScript-Exceptions und WebSocket-Verbindungsabbrüche. Klassische Sättigungsmetriken wie CPU und Speicher beziehen sich auf das Endgerät und liegen außerhalb des Einflussbereichs des Betriebsteams. Da die Grenzen der Frontend-Beobachtbarkeit eng gesteckt sind, liegt der instrumentierungstechnische Schwerpunkt auf den serverseitigen Komponenten.

6.3.2 Spring-Boot-Backend

Das Backend ist die zentrale Verarbeitungskomponente und der in Kapitel 3 identifizierte Single Point of Failure. Der primäre Latenzindikator ist die Antwortdauer je REST-Endpunkt als Histogramm (p50, p95, p99), weil Durchschnittswerte die Erfahrung langsam bedienter Nutzer verbergen können (Ewaschuk, 2017, S. 7). Ebenso muss die Latenz erfolgreicher und fehlgeschlagener Anfragen getrennt erfasst werden, da schnell zurückgegebene Fehler den Gesamtdurchschnitt verzerren. Ergänzend werden die Latenzen ausgehender Aufrufe an PostgreSQL, den Message Broker und den AI Provider separat erhoben. Das reaktive Programmiermodell auf Basis von WebFlux erfordert dabei besondere Aufmerksamkeit, da die Messung auf Span-Ebene innerhalb der reaktiven Kette erfolgen muss (Sundström, 2025, S. 25).

Die Anfragerate pro Endpunkt bildet den Traffic ab. Für die geschäftskritischen Abläufe aus Abschnitt 3.3 ist eine gesonderte Erfassung sinnvoll. Fehler werden auf mehreren Ebenen erfasst: HTTP-5xx-Statuscodes, unbehandelte Exceptions und fehlgeschlagene Downstream-Aufrufe. Da kein externer Aufruf einen Timeout besitzt (R1), äußern sich Timeout-bedingte Fehler als hängende Anfragen, was die Latenzüberwachung umso dringlicher macht. Die Sättigung wird über Container-Metriken und den R2DBC-Connection-Pool gemessen. Ewaschuk (2017, S. 8) empfiehlt, Sättigung indirekt über steigende Latenz am 99. Perzentil zu erkennen, weil Latenzzunahmen häufig ein Frühindikator für Ressourcenerschöpfung sind.

6.3.3 PostgreSQL

Die Datenbank ist an allen drei geschäftskritischen Nutzungspfaden beteiligt. Die Vote-Trigger und die Comment-Nummern-Generierung (Abschnitt 3.3) erzeugen potenzielle Engpässe. Der

zentrale Latenzindikator ist die Abfrageausführungszeit, wobei lang laufende Abfragen auf fehlende Indizes oder Row-Level Contention hindeuten. Die Transaktionsrate und die aktive Verbindungszahl bilden den Traffic ab. Relevante Fehlersignale umfassen Deadlocks, fehlgeschlagene Transaktionen und Verbindungsablehnungen. Die Sättigung wird über das Verhältnis genutzter zu maximal verfügbarer Verbindungen, die Buffer-Cache-Hit-Rate und die Disk-I/O-Auslastung gemessen. Ein sinkender Cache-Hit-Anteil deutet darauf hin, dass die Arbeitsspeicherkonfiguration nicht mehr zum Datenvolumen passt.

6.3.4 Externe KI-Dienste

Die Beobachtung der externen KI-Dienste beschränkt sich auf die Schnittstelle zwischen AI Provider und den angebundenen Modellen, da der Quellcode der LLM-Provider nicht verfügbar ist (Bissig, 2024, S. 31). Die LLM-Antwortzeit wird als Histogramm pro Provider und Modell erfasst. Da kein Timeout konfiguriert ist (R1), kann ein einzelner hängender Aufruf den AI Provider blockieren und über die Aufrufkette das Backend beeinflussen. Der Traffic wird als Anzahl der LLM-Aufrufe pro Zeiteinheit je Funktionstyp gemessen. Fehlersignale sind HTTP-Fehlerantworten der Provider-APIs (insbesondere 429 Rate Limit Exceeded), Timeouts und Parsing-Fehler. Neun von 19 LLM-Providern sind explizit mit `max_retries=0` konfiguriert, sodass ein fehlgeschlagener Aufruf unmittelbar durchschlägt. Da die LLM-Provider keinen Einblick in ihre Ressourcenauslastung gewähren, wird Sättigung über Proxy-Indikatoren wie steigende Antwortzeiten und zunehmende Rate-Limit-Fehler abgebildet.

6.4 Serviceübergreifende Signal-Matrix

Tabelle 6.1 verdichtet die vorangehenden Ableitungen in einer kompakten Gesamtübersicht. Sie bildet die Grundlage für das rollenbasierte Dashboard-Design (Abschnitt 6.5) und definiert den Rahmen für das Alerting-Konzept (Abschnitt 6.6), weil nicht jede Messgröße als Alarmauslöser geeignet ist.

Tabelle 6.1: Serviceübergreifende Signal-Matrix mit Zuordnung von Dienst, Golden Signal, konkreter Messgröße, Erhebungszweck und primärer Stakeholder-Rolle. Quelle: Eigene Darstellung.

Dienst	Signal	Messgröße	Zweck	Rolle
Backend	Latency	Antwortdauer je Endpunkt (p50, p95, p99)	Nutzererfahrung, Engpasserkennung	Entwicklung
Backend	Traffic	Anfragen/s je Pfad, Broker-Nachrichten/s	Lastbild, Kapazitätsplanung	DevOps
Backend	Errors	HTTP-5xx-Rate, Exception-Rate, Downstream-Fehler	Fehlererkennung, Trendanalyse	Entwicklung
Backend	Saturation	CPU, Speicher, R2DBC-Pool-Nutzung	Ressourcenwarnung	DevOps
PostgreSQL	Latency	Abfrageausführungszeit (Durchschnitt, Maximum)	Indexbedarf, Contention-Erkennung	Entwicklung
PostgreSQL	Traffic	Transaktionen/s, aktive Verbindungen	Nutzungsintensität	DevOps
PostgreSQL	Errors	Deadlocks, fehlgeschl. Transaktionen, Lock-Waits	Stabilitätsüberwachung	Entwicklung
PostgreSQL	Saturation	Verbindungspool-Quote, Cache-Hit-Rate, Disk-I/O	Kapazitätswarnung	DevOps
AI Prov.	Latency	LLM-Antwortzeit je Provider/Modell (Histogramm)	SLA-Proxy, Kaskadenerkennung	Entwicklung
AI Prov.	Traffic	LLM-Aufrufe/Zeiteinheit je Funktionstyp	Nutzungsmuster, Kontingentplanung	Product Owner
AI Prov.	Errors	Provider-Fehlerrate, 429er-Rate, Parsing-Fehler	Verfügbarkeit ext. Abhängigkeiten	Entwicklung
AI Prov.	Saturation	Antwortzeit-Trend, Rate-Limit-Häufigkeit, lokale CPU	Kapazitätswarnung	DevOps
Frontend	Latency	API-Aufruf-Dauer, Raum-Ladezeit	Nutzererfahrung	Product Owner
Frontend	Traffic	Aktive WS-Verbindungen, HTTP-Anfragen/s	Nutzungsgrad	Product Owner
Frontend	Errors	HTTP-Fehler, JS-Exceptions, WS-Abbrüche	Fehlererkennung clientseitig	Entwicklung
Frontend	Saturation	Antwortzeit-Trend, Retry-Häufigkeit	Überlastindikation	DevOps

6.5 Rollenbasiertes Informations- und Dashboard-Konzept

Für frag.jetzt wird eine hierarchische Informationsarchitektur in drei Sichtebenen umgesetzt, deren Zuschnitt sich aus den Stakeholder-Gruppen und der Signal-Matrix ableitet (Sakinala, 2025, S. 108).

Übersichtssicht (Product Owner / Management). Diese Ebene beantwortet die Frage, ob frag.jetzt aus Nutzersicht funktioniert. Sie zeigt verdichtete Indikatoren wie aktive Raumsitzungen, globale Fehlerrate und Gesamtlatenz der Kernabläufe. Detailinformationen sind nur über Verlinkung erreichbar.

Dienst- und Infrastruktursicht (DevOps / Betrieb). Diese Ebene beantwortet die Frage, wo das Problem liegt. Sie stellt Ressourcenmetriken je Container, Datenbankauslastung, Message-Broker-Kennzahlen und eine aus Trace-Daten gewonnene Service-Map dar.

Diagnose- und Entwicklungssicht (Entwicklung). Diese Ebene beantwortet die Frage, was genau im Code passiert. Sie bietet endpunktbezogene Latenzhistogramme, Fehleraufschlüsselungen, Trace-Ansichten und die Möglichkeit, aus einem Trace in die zugehörigen Logeinträge zu springen. Die technische Umsetzung wird in Kapitel 7 dokumentiert.

6.6 Alerting-Konzept und Runbook-Logik

Aufbauend auf den in Abschnitt 2.7 erarbeiteten Grundlagen folgt das Alerting-Konzept drei Grundsätzen: Alarme reagieren symptomorientiert auf nutzerwirksame Zustände statt auf interne Ursachenindikatoren (Ewaschuk, 2017, S. 11–12). Jeder Alarm muss handlungsfähig sein, also Problem, betroffene Komponente und empfohlene Erstmaßnahme erkennbar machen (Vinnakota & Kolla, 2025, S. 174). Schweregrade orientieren sich an der Nutzerwirkung: *Critical* für akute Ausfälle geschäftskritischer Funktionen, *Warning* für sich abzeichnende Verschlechterungen.

6.6.1 Alarmkategorien

Aus der Signal-Matrix lassen sich vier Kategorien ableiten. Die konkreten Schwellenwerte und Zeitfenster werden in Kapitel 7 festgelegt.

Verfügbarkeitsalarme (Critical). Ein Alarm wird ausgelöst, wenn eine geschäftskritische Funktion nicht mehr erreichbar ist, etwa wenn das Backend über einen definierten Zeitraum keine erfolgreichen Antworten liefert oder die Datenbankverbindung dauerhaft fehlschlägt.

Latenzalarme (Warning / Critical). Steigt die Antwortzeit eines Kernendpunkts über einen definierten Grenzwert, etwa das 95. Perzentil über ein gleitendes Fenster, wird zunächst eine Warnung und bei weiterer Verschlechterung ein kritischer Alarm erzeugt. Für die KI-Dienste gelten angepasste Grenzwerte.

Fehleralarme (Warning / Critical). Eine Fehlerrate oberhalb eines definierten Anteils löst eine Warnung aus. Die explizite Trennung von client- (4xx) und serverseitigen (5xx) Fehlern verhindert, dass ungültige Eingaben denselben Alarm wie interne Serverfehler erzeugen.

Sättigungsalarme (Warning). Ressourcenmetriken wie CPU-Auslastung und Datenbankverbindungs-pool-Nutzung werden als vorausschauende Warnung konfiguriert. Ewaschuk (2017, S. 8) empfiehlt, Sättigung indirekt über steigende Latenz am 99. Perzentil zu erkennen.

6.6.2 Runbook-Verknüpfung

Jeder Critical-Alarm wird mit einem Runbook verknüpft, das Problembeschreibung, betroffene Komponente, Diagnoseschritte, Erstmaßnahmen und Eskalationspfad enthält. Für den Prototyp werden exemplarische Runbooks für den Kaskadeneffekt durch KI-Service-Timeouts (R1) und Datenbankengpässe bei Massenabstimmungen (R2) erstellt.

Regelbasierte Alarme stoßen an Grenzen, sobald sich Lastmuster im Tages- oder Semesterverlauf verändern und starre Schwellenwerte entweder zu viele False Positives oder zu späte Warnungen produzieren (Vinnakota & Kolla, 2025, S. 173). Anomaliebasierte Verfahren setzen jedoch ein trainiertes Modell des Normalverhaltens voraus, das für frag.jetzt mangels historischer Produktivdaten derzeit nicht existiert (Soldani & Brogi, 2022, S. 5–6). Die standardisierte Metrikerhebung über Prometheus und die offene Abfrageschnittstelle über PromQL erlauben es, solche Verfahren als zukünftigen Ausbauschritt ohne Änderung der Instrumentierung anzubinden.

7 Prototypische Implementierung

Dieses Kapitel dokumentiert die praktische Umsetzung der in Kapitel 6 konzipierten Observability-Strategie und beschreibt die Instrumentierung der Kernkomponenten, exemplarische Validierungsszenarien, das Golden-Signals-Dashboard, die Alerting-Regeln sowie aufgetretene Herausforderungen.

7.1 Umfang und Abgrenzung der prototypischen Umsetzung

Die Umsetzung erfolgte auf dem Staging-Server von frag.jetzt, auf dem der vollständige Anwendungsstack in einer produktionsnahen Docker-Compose-Konfiguration betrieben wird. Der Observability-Stack wurde als eigenständiges Compose-Projekt deployt und über ein gemeinsames Docker-Netzwerk an die Applikationscontainer angebunden.

Die Implementierung umfasst drei Schwerpunkte: das Deployment des LGTM-Stacks mit allen Speicher-Backends und Infrastruktur-Exportern, die Instrumentierung von Backend und WS-Gateway über den OpenTelemetry Java Agent und die Erstellung eines Golden-Signals-Dashboards mit zugehörigen Alerting-Regeln. Bewusst ausgeklammert wurden die Instrumentierung des AI Providers und des Frontends, die clientseitige Browser-Telemetrie, die Produktivhärtung sowie die rollenspezifischen Dashboard-Sichten aus Kapitel 6.5. Eine durchgängige Ende-zu-Ende-Beobachtbarkeit aller Systemgrenzen kann daher nicht nachgewiesen werden. Die Teilumsetzung erlaubt jedoch die Prüfung der zentralen Mechanismen (vgl. Bissig, 2024, S. 52).

7.2 Instrumentierung ausgewählter Kernkomponenten

Die Instrumentierung folgt dem in Kapitel 6.2 beschriebenen Schichtenmodell. Tabelle 7.1 gibt eine Übersicht über die instrumentierten Komponenten, die erzeugten Signale und den im Prototyp beobachteten Nachweis.

7.2.1 Observability-Stack und Telemetrie-Pipeline

Das Deployment umfasst acht Container. Der OpenTelemetry Collector empfängt sämtliche Applikationstelemetrie über OTLP und verteilt sie an die drei Speicher-Backends: Metriken an Prometheus, Logs an Loki und Traces an Tempo. Für die Infrastrukturschicht erfassen cAdvisor, der Node Exporter und der PostgreSQL Exporter die jeweiligen Metriken. Alle Observability-Container sind mit expliziten Speicherlimits konfiguriert, um das Risiko einer unkontrollierten Ressourcenverdrängung (R5) nicht im Observability-Stack selbst zu reproduzieren. Grafana wird mit provisionierten Datenquellen gestartet, die insbesondere die Verknüpfungen zwischen den Backends herstellen: Tempo-Traces können auf korrespondierende Loki-Logs verweisen und umgekehrt.

Tabelle 7.1: Übersicht der instrumentierten Komponenten und beobachteten Telemetriesignale.
Quelle: Eigene Darstellung.

Komponente	Instrumentierung	Erzeugte Signale	Nachweis im Prototyp
Backend (Spring Boot)	OTel Java Agent (Auto-Instr.)	HTTP-Spans, DB-Spans, Logback-Logs mit Trace-ID, JVM-Metriken	Span-Waterfall mit DB-Operationen in Grafana sichtbar
WS-Gateway (Netty)	OTel Java Agent (Auto-Instr.)	WebSocket-Spans, RabbitMQ-Spans, JVM-Metriken	Service Graph zeigt Knoten mit Anfragerate
PostgreSQL	PostgreSQL Exporter	Verbindungszähler, Transaktionsraten, Lock-Statistiken	<code>pg_stat_activity_count</code> in Prometheus abfragbar
RabbitMQ	Prometheus-Plugin	Queue-Tiefen, Nachrichtenraten	<code>rabbitmq_queue_messages</code> in Dashboard-Panel sichtbar
Host	Node Exporter, cAdvisor	CPU, Speicher, Netzwerk, Container-Metriken	Gauge-Panels im Dashboard mit Schwellenwertfärbung

7.2.2 Applikationsinstrumentierung über den OTel Java Agent

Für die JVM-basierten Dienste wurde der OpenTelemetry Java Agent in Version 2.26.0 eingesetzt. Der Agent wird als Read-Only-Volume in den Container gemountet und über Umgebungsvariablen im Docker-Compose-Override-File konfiguriert. Dieser Ansatz vermeidet Änderungen am Applikationscode und an den bestehenden Docker-Images.

Der Agent instrumentiert automatisch HTTP-Anfragen (Spring WebFlux, Netty), R2DBC-Datenbankzugriffe, RabbitMQ-Interaktionen und Logback-Logausgaben. Im Ergebnis erzeugt jeder eingehende HTTP-Request einen Root-Span, der untergeordnete Spans für Datenbankoperationen enthält. Für die instrumentierten Dienste ist damit der Verarbeitungspfad vom HTTP-Eingang bis zur Datenpersistierung in einem zusammenhängenden Trace rekonstruierbar, was Anforderung A1 in Teilen adressiert.

Der Service Graph in Grafana visualisiert die Kommunikationsbeziehungen zwischen den instrumentierten Diensten und zeigt im Prototyp drei Knoten (`fragjetzt-backend`, `fragjetzt-ws-gateway` und `fragjetzt-postgres`) mit ihren jeweiligen Anfrageraten und Antwortzeiten.

7.2.3 Infrastruktorexporter und Scrape-Konfiguration

Prometheus scrapet sechs Targets im 15-Sekunden-Intervall. Die Infrastruktur-Exporter liefern Metriken, die direkt auf die Sättigungsindikatoren der Signal-Matrix aus Kapitel 6.4 abbilden und damit Anforderung A5 auf Metrikebene im Prototyp adressieren. Ergänzend empfängt Prometheus über den OTLP-Receiver Applikationsmetriken vom OTel Collector. Tempo generiert aus den eingehenden Traces automatisch Span-Metriken und schreibt diese per Remote Write

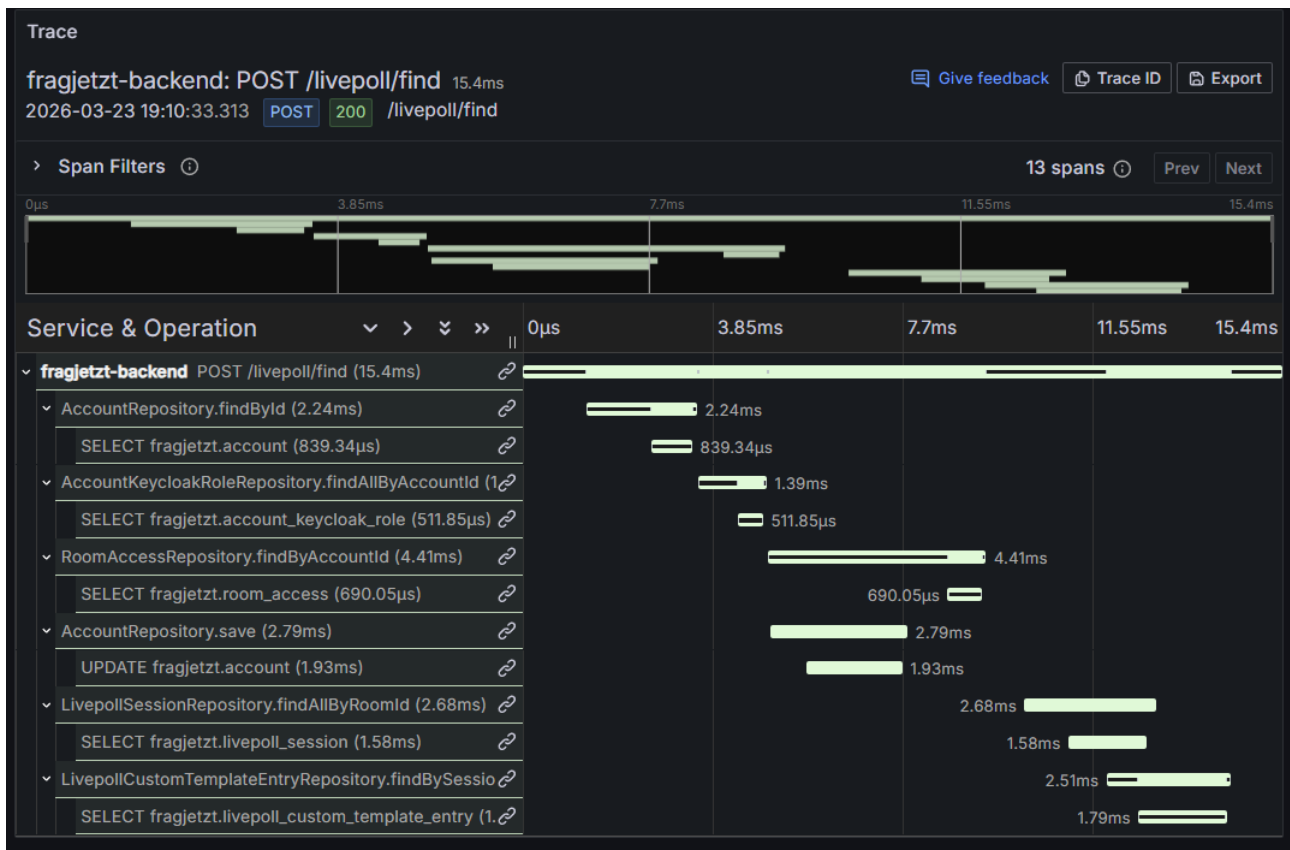


Abbildung 7.1: Span-Waterfall eines Backend-Traces (`POST /livepoll/find`) mit 13 Spans und einer Gesamtdauer von 15,4 ms. Quelle: Eigene Darstellung.

an Prometheus zurück. Aus diesen Span-Metriken werden der Service Graph und die Traffic- sowie Fehlerratenpanels im Dashboard gespeist.

7.2.4 Logging-Anbindung

Das Backend erzeugte im Standardbetrieb nahezu keine Logausgaben, weil Spring Boot WebFlux eingehende HTTP-Requests nicht automatisch protokolliert. Für den Prototyp wurde Debug-Level-Logging für die HTTP-Verarbeitungsschicht selektiv aktiviert, sodass Request-bezogene Einträge in Loki verfügbar sind und über die vom OTel Agent automatisch injizierten Trace-IDs mit den zugehörigen Spans korreliert werden können. Diese Korrelierbarkeit adressiert Anforderung A3 in Grundzügen. Für einen Produktiveinsatz wäre strukturierte JSON-Protokollierung mit selektiver Schweregradfilterung vorzuziehen.

7.2.5 Exemplarische Validierungsszenarien

Um die Funktionsfähigkeit der Telemetrie-Pipeline zu belegen, wurden drei Szenarien auf dem Staging-Server durchgeführt.

Szenario 1: Normaler Nutzerrequest (Frage stellen). Ein Nutzer erstellte über die Weboberfläche eine neue Frage. In Grafana war der entsprechende Trace mit 8 Spans sichtbar, darunter

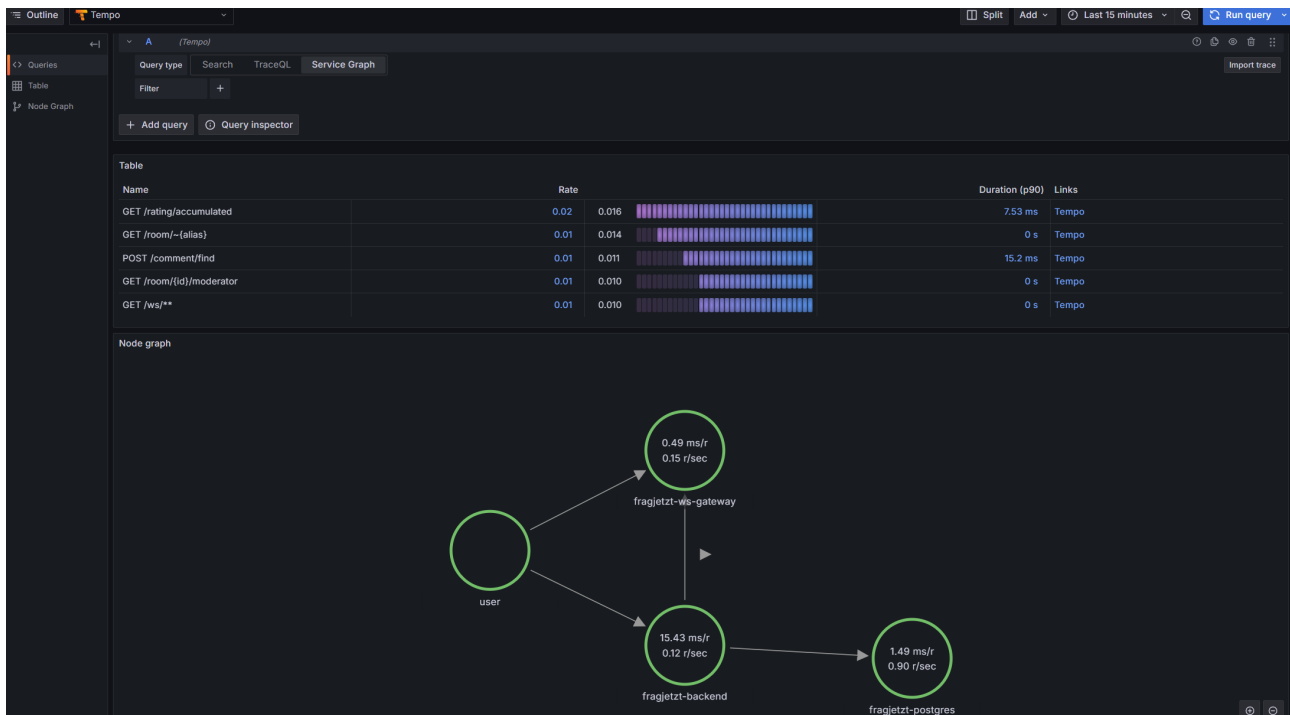


Abbildung 7.2: Service Graph aus Tempo mit den Kommunikationspfaden user → fragjetzt-backend → fragjetzt-postgres sowie user → fragjetzt-ws-gateway. Quelle: Eigene Darstellung.

der INSERT-Span auf der PostgreSQL-Datenbank. Das Latenz-Panel zeigte den Request im p50-Bereich bei circa 12 ms.

Szenario 2: Erhöhte Last auf Voting-Endpunkt. Durch wiederholte manuelle Abstimmungen wurde eine erhöhte Anfragerate erzeugt. Im Dashboard stieg die Anfragerate erkennbar an. Der PostgreSQL-Verbindungszähler blieb im unkritischen Bereich, was bei manueller Last erwartbar ist und ein automatisiertes Lastwerkzeug erfordern würde.

Szenario 3: Provozierter Fehlerfall. Ein Request an einen nicht existierenden Endpunkt erzeugte einen 404-Statuscode. In Tempo war der zugehörige Trace mit Fehler-Tag sichtbar, im Errors-Panel erschien der Endpunkt in der Tabelle der fehlerhäufigsten Span-Operationen. Der 5xx-Alert wurde erwartungsgemäß nicht ausgelöst.

7.3 Umsetzung des Golden-Signals-Dashboards

Das Dashboard enthält 15 Panels in fünf Sektionen. Die Panelstruktur bildet die Signal-Matrix aus Kapitel 6.4 ab.

Die Sektion *Latency* zeigt Backend-Antwortzeiten als Perzentil-Zeitreihen (p50, p95, p99) und eine aufgeschlüsselte Latenzansicht pro Endpunkt, basierend auf `http_server_request_duration_seconds`. Die Sektion *Traffic* stellt die Anfragerate pro Dienst aus Span-Metriken sowie die meistfrequentierten Endpunkte als gestapeltes Balkendiagramm dar. Die Sektion *Errors* kombiniert eine Span-basierte Fehlerrate pro Service mit einem Stat-Panel für die HTTP-5xx-Quote

und einer Tabelle der fehlerhäufigsten Span-Operationen. Die Sektion *Saturation* umfasst JVM-Heap-Auslastung, Thread-Zähler und CPU-Nutzung als Gauge-Panel mit Schwellenwertfärbung. Die *Infrastruktur*-Sektion zeigt PostgreSQL-Verbindungen, RabbitMQ-Queue-Tiefen, Host-Ressourcen, GC-Pausen und den Service Graph aus Tempo.

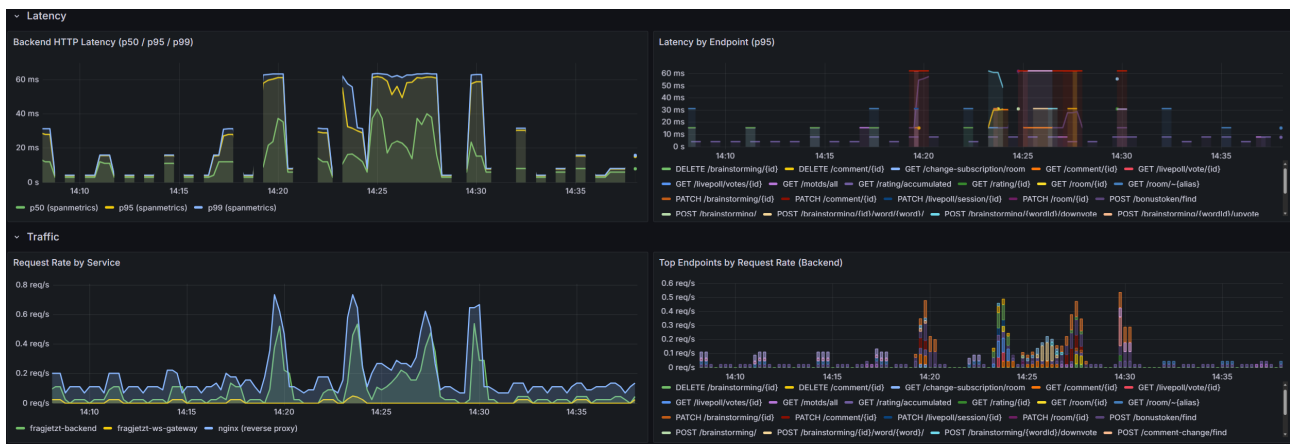


Abbildung 7.3: Ausschnitt des Golden-Signals-Dashboards mit den Sektionen *Latency* und *Traffic*. Quelle: Eigene Darstellung.

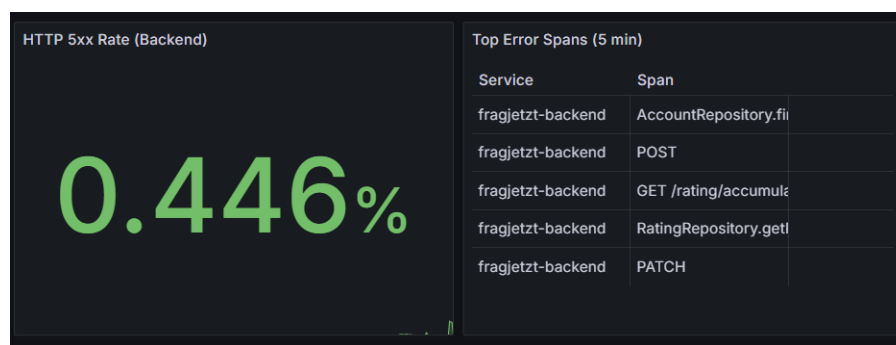


Abbildung 7.4: Ausschnitt der *Errors*-Sektion mit Dummy-Fehlerrate zur Veranschaulichung der Fehlerindikatoren. Quelle: Eigene Darstellung.

Sämtliche Panels verwenden `$_rate_interval` als dynamisches Zeitfenster für Rate-Berechnungen. Die vollständige JSON-Definition befindet sich in Anhang [VERWEIS].

7.4 Alerting-Regeln und Beispiel-Runbooks

Die Alerting-Logik aus Kapitel 6.6 wird exemplarisch in drei Grafana-Alerting-Regeln überführt, die Anforderung A4 adressieren.

Die erste Regel (*Critical*) löst aus, wenn die HTTP-5xx-Fehlerrate des Backends über fünf Minuten den Schwellenwert von 5 % überschreitet. Die zweite Regel (*Warning*) reagiert auf eine PostgreSQL-Verbindungsauslastung über 70 % des konfigurierten Limits von 100 Verbindungen. Die dritte Regel (*Warning*) überwacht die Backend-Latenz im p95-Bereich und alarmiert bei einer anhaltenden Überschreitung von 500 ms über drei Minuten.

Die Evaluationsfenster von drei bis fünf Minuten fungieren als Debounce-Mechanismus, der

transiente Spitzen toleriert (Vinnakota & Kolla, 2025, S. 176). Jede Regel enthält in ihrer Annotation-Konfiguration einen Link zum zugehörigen Runbook sowie zum relevanten Dashboard-Panel. Die drei Regeln wurden im Prototyp angelegt und konfiguriert, eine systematische Verifikation durch gezielte Störungsinjektionen steht noch aus.

Ergänzend wurden zwei Runbooks als strukturierte Markdown-Dokumente erstellt, die der Fünf-Felder-Struktur (Symptom, Kontext, Diagnose, Behebung, Eskalation) aus Kapitel 6.6 folgen. Das erste behandelt den Alarm “Backend-Fehlerrate über Schwellenwert”, das zweite den Alarm “PostgreSQL-Verbindungspool nahe Limit”. Beide Runbooks befinden sich im Anhang [VERWEIS].

7.5 Herausforderungen und Lösungen bei der Umsetzung

Drei Kategorien von Herausforderungen traten auf, die in vergleichbaren Deployments ebenfalls zu erwarten wären.

7.5.1 Ressourcendimensionierung

Tempo wurde initial mit einem Speicherlimit von 256 MB konfiguriert. Unter der Trace-Last erschöpfte der Container seinen verfügbaren Speicher und wurde wiederholt durch den Kernel beendet (OOM-Kill). Jeder Neustart hinterließ unvollständige WAL-Dateien, die beim nächsten Start erneut Fehler verursachten. Die Lösung bestand in der Erhöhung des Speicherlimits auf 512 MB und der Bereinigung des WAL-Volumes. Das Problem veranschaulicht, dass die Ressourcendimensionierung von Observability-Komponenten auf einem Single-Host-System sorgfältige Abstimmung erfordert (Bissig, 2024, S. 52).

7.5.2 Netzwerktopologie und Dienstkommunikation

Tempo generiert aus eingehenden Traces Span-Metriken, die über Remote Write an Prometheus übermittelt werden und den Service Graph in Grafana speisen. Im initialen Deployment scheiterte dieser Export an der Netzwerktrennung zwischen den Compose-Projekten. Die Lösung bestand darin, Prometheus in beiden Docker-Netzwerken verfügbar zu machen. Anders als in Kubernetes, wo Pods standardmäßig über ein flaches Netzwerk kommunizieren, erfordern separate Compose-Projekte explizite Netzwerkkonfigurationen.

7.5.3 Signalqualität und Rauschunterdrückung

Die Auto-Instrumentierung erfasst alle erkennbaren Operationen, ohne zwischen geschäftsrelevanten und internen Abläufen zu unterscheiden. Periodische Hintergrundprozesse wie der `ScheduledMessageSender` dominierten die Trace-Liste und machten diagnostisch relevante Traces kaum auffindbar. Statt Sampling, das auch relevante Daten verlieren kann, wurden

gezielt Span-Namen ohne diagnostischen Wert auf Collector-Ebene über einen `filter/traces`-Processor verworfen.

Tabelle 7.2 ordnet die umgesetzten Komponenten den Anforderungen aus Kapitel 3 zu und benennt die jeweiligen Einschränkungen.

Tabelle 7.2: Zuordnung der prototypischen Umsetzung zu den Observability-Anforderungen.
Quelle: Eigene Darstellung.

Anf.	Umsetzung im Prototyp	Einschränkung
A1	Distributed Tracing über OTel Agent; Span-Waterfall und Service Graph in Grafana; Trace-basierte Validierung in Szenario 1	Nur Backend und WS-Gateway instrumentiert; AI Provider und Frontend nicht abgedeckt
A2	Applikations- und Infrastrukturmetriken in Prometheus; Dashboard-Sektionen entlang der vier Golden Signals	Keine Browser-Telemetrie; Frontend-Ebene fehlt
A3	Log-Weiterleitung an Loki über OTel Agent mit automatischer Trace-ID-Injektion	Debug-Level-Logging als Demonstrationskonfiguration; kein strukturiertes JSON-Format
A4	Drei symptomorientierte Alerting-Regeln mit Debounce-Logik; zwei Runbooks mit Fünf-Felder-Struktur; 5xx-Alert in Störungsszenario ausgelöst	Kurzfristige Fehler-Metriken lieferten keine konsistent aktuellen Werte; keine dynamischen Schwellenwerte
A5	cAdvisor, Node Exporter, PostgreSQL Exporter, RabbitMQ-Plugin; Infrastrukturektion im Dashboard	Im Prototyp für die vorhandene Infrastruktur abgebildet

8 Evaluation der entwickelten Strategie

Das vorliegende Kapitel prüft, inwieweit die in Kapitel 6 konzipierte und in Kapitel 7 prototypisch umgesetzte Observability-Strategie die Anforderungen aus Kapitel 3 erfüllt.

8.1 Evaluationsmethodik

Die Evaluation folgt dem in Abschnitt 4.3 definierten Bewertungsrahmen mit drei Evidenzquellen: Artefaktanalyse, Konfigurations- und Instrumentierungsprüfung sowie Last- und Störungsszenarien. Im Prototyp (Kapitel 7) wurden die ersten beiden Quellen ausgeschöpft. Die dritte blieb auf manuelle Plausibilitätstests beschränkt. Um diese Lücke zu verkleinern, wurden drei gezielte Belastungsszenarien auf dem Staging-Server durchgeführt: CPU-Last auf dem Backend-Container für den Latenz-Alert, temporäre Unterbrechung der Datenbankverbindung für den Fehler-Alert und Erschöpfung des PostgreSQL-Verbindungspools für den Sättigungs-Alert. Ergänzend wurde die bidirektionale Log-Trace-Korrelation auf dem Staging-Server verifiziert.

8.2 Anforderungsbezogene Bewertung

Tabelle 8.1 gibt vorab einen kompakten Überblick über die Erfüllungsgrade. Die folgenden Unterabschnitte begründen diese Einstufungen.

Tabelle 8.1: Ergebnissynthese der anforderungsbezogenen Evaluation. Quelle: Eigene Darstellung.

Anf.	Kurzbefund	Wesentliche Einschränkung	Erfüllungsgrad
A1	Backend-Pfad rekonstruierbar, Gesamtsystem unvollständig	AI Provider und Frontend nicht instrumentiert	teilweise erfüllt
A2	Drei Messebenen für instrumentierte Dienste vorhanden	Keine Browser-Telemetrie	weitgehend erfüllt
A3	Bidirektionale Log-Trace-Korrelation validiert	Debug-Level-Logging, noch nicht produktionsorientiert strukturiert	erfüllt
A4	Latenz-Alert validiert, Fehler-Alert ausgelöst, Sättigungs-Alert metrisch vorbereitet	Kurzfristige Fehler-Metriken lieferten keine konsistent aktuellen Werte; Sättigungs-Alert nicht experimentell getestet	teilweise erfüllt
A5	Infrastrukturmetriken verfügbar und plausibel	Nicht alle Indikatoren unter Last geprüft	weitgehend erfüllt

8.2.1 A1: Ende-zu-Ende-Nachvollziehbarkeit

Prüffrage: Lässt sich ein Request Ende-zu-Ende rekonstruieren?

Die Instrumentierung über den OTEL Java Agent erzeugt für jeden eingehenden HTTP-Request einen Root-Span mit untergeordneten Spans für Datenbankoperationen und interne Verarbeitungsschritte. Abbildung 8.1 zeigt exemplarisch einen Trace für den Endpunkt GET /room/{id}/moderator mit 11 Spans, darunter mehrere SELECT-Operationen auf der PostgreSQL-Datenbank. Der Verarbeitungspfad vom HTTP-Eingang bis zur Datenpersistierung ist vollständig rekonstruierbar. Der Service Graph in Grafana bildet die Kommunikationsbeziehungen zwischen den drei instrumentierten Diensten ab.

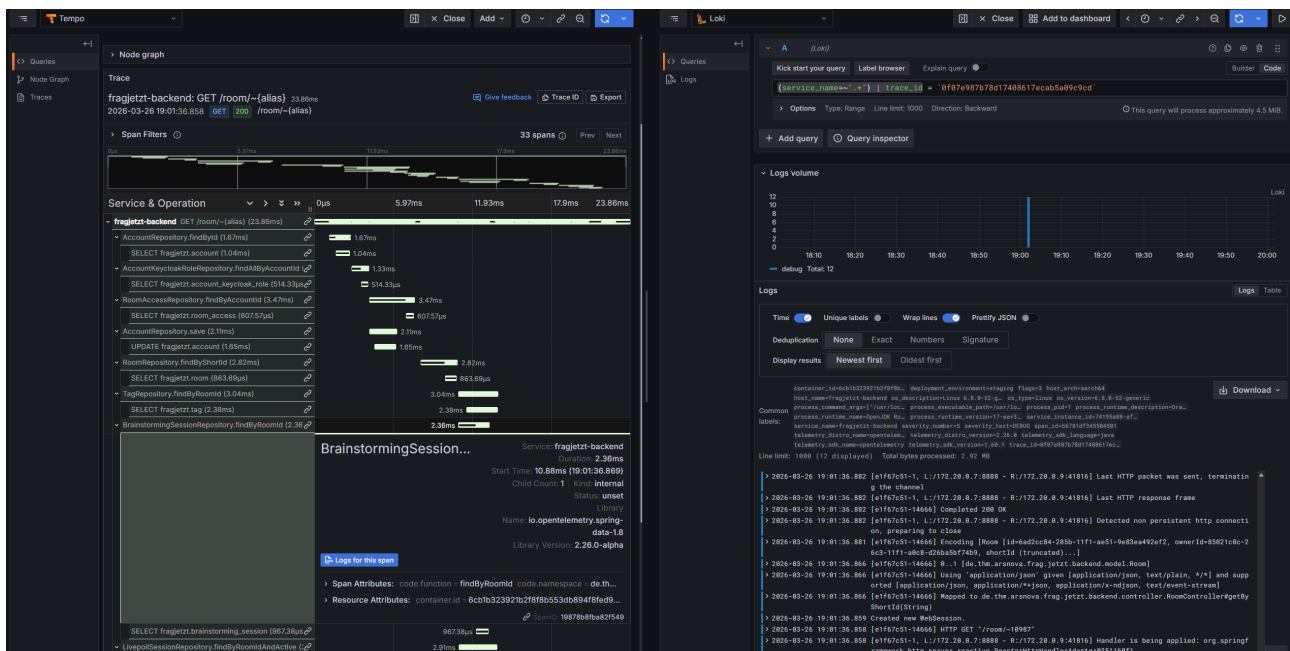


Abbildung 8.1: Trace-Waterfall in Tempo für den Endpunkt GET /room/{id}/moderator mit 11 Spans und zugeordneten Datenbankoperationen. Der rechte Bereich zeigt die korrelierten Loki-Logs, erreichbar über die Funktion *Logs for this span*. Quelle: Eigene Darstellung (Screenshots, Staging-Server).

Die Nachvollziehbarkeit endet an den Grenzen der instrumentierten Komponenten. Der AI Provider und das Frontend sind nicht instrumentiert. Für den instrumentierten Backend-Pfad ist die Request-Rekonstruktion funktional gegeben, für das Gesamtsystem bleibt sie unvollständig. Der Erfüllungsgrad wird daher als *teilweise erfüllt* bewertet.

8.2.2 A2: Mehrstufige Messbarkeit entlang der Golden Signals

Prüffrage: Deckt die Instrumentierung alle drei Messebenen ab?

Das Golden-Signals-Dashboard bildet alle vier Dimensionen in separaten Sektionen ab. Im Belastungsszenario stieg die p95-Latenz unter CPU-Last von rund 20 ms auf über 500 ms, was im Dashboard sofort sichtbar wurde. Nach dem temporären Stopp der Datenbank stieg die 5xx-Rate und die betroffenen Endpunkte erschienen in der Fehlertabelle. Die drei Messebenen,

Applikationsmetriken über den OTel Agent, Infrastrukturmetriken über Exporter und Trace-basierte Span-Metriken über Tempo, sind für die instrumentierten Dienste abgedeckt. Die Lücke betrifft die fehlende Frontend-Telemetrie. Der Erfüllungsgrad wird als *weitgehend erfüllt* eingestuft.

8.2.3 A3: Strukturiertes, korrelierbares Logging

Prüffrage: Können Logeinträge über Korrelations-IDs verknüpft werden?

Die bidirektionale Korrelation zwischen Logs und Traces konnte auf dem Staging-Server nachgewiesen werden. Von einem Trace in Tempo führt die Funktion *Logs for this span* zu einer Loki-Abfrage, die nach Trace-ID und Dienstname filtert. In der Gegenrichtung enthält jeder Log-Eintrag in Loki ein klickbares *TraceID*-Feld, das direkt zum zugehörigen Trace navigiert. Diese Verknüpfung wurde über ein Derived Field in der Loki-Datenquellenkonfiguration realisiert.

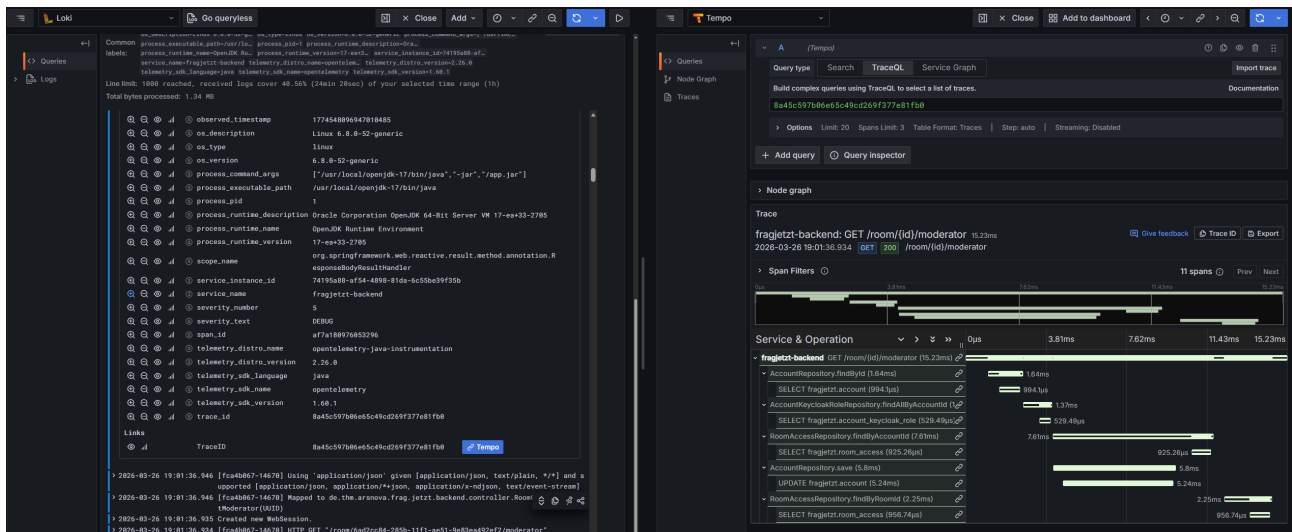


Abbildung 8.2: Log-to-Trace-Korrelation in Loki: Ein Logeintrag mit klickbarem TraceID-Link (links) navigiert zum zugehörigen Trace-Waterfall in Tempo (rechts). Quelle: Eigene Darstellung (Screenshot, Staging-Server).

Da die Kernfunktion der Korrelierbarkeit nachgewiesen wurde, wird A3 als *erfüllt* bewertet. Für den Produktivbetrieb wäre eine Reduktion auf strukturierte JSON-Protokollierung erforderlich.

8.2.4 A4: Handlungsfähige Alarmierung

Prüffrage: Reagieren Alarme auf nutzerwirksame Zustände und enthalten sie ausreichend Kontext für eine Erstreaktion?

Der Latenz-Alert wurde durch künstlich erzeugten CPU-Druck auf den Backend-Container im Zustand *Firing* ausgelöst. Abbildung 8.3 zeigt den Alarm mit Symptombeschreibung, Prüfpfad und Runbook-Link in den Annotations. Die Multi-Signal-Logik bewährte sich: In Leerlaufphasen ohne aktive Requests verhinderte die Mindest-Traffic-Rate von 0,1 req/s, dass der Alert fälschlicherweise auslöst.

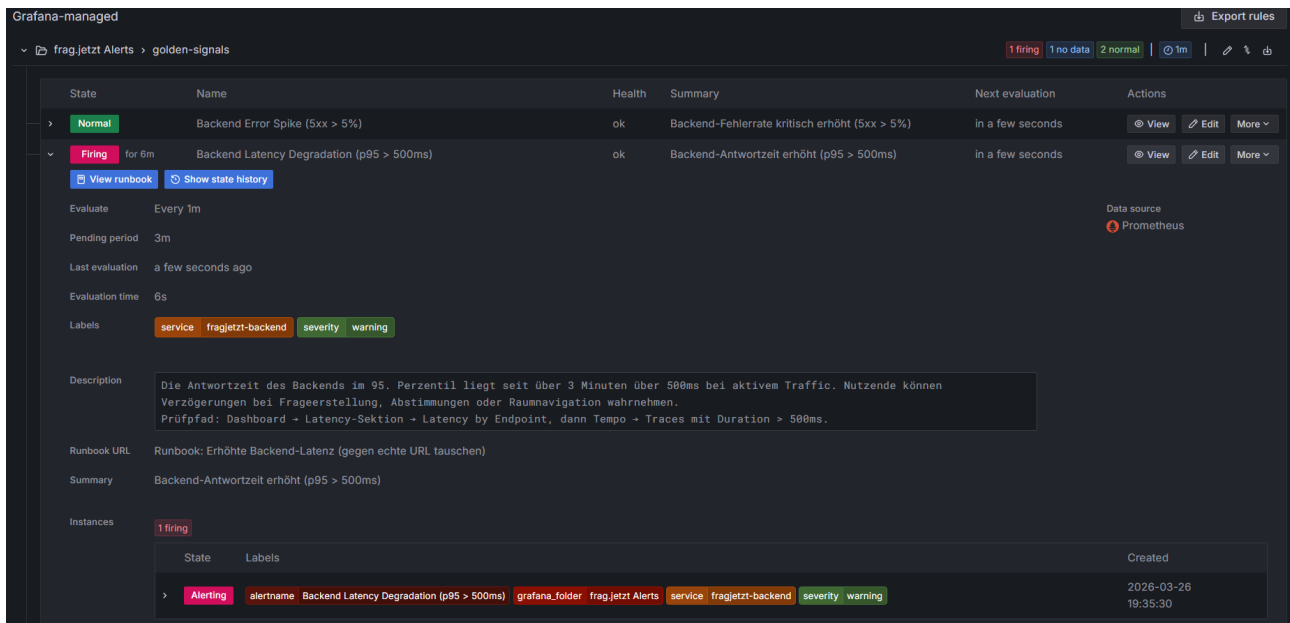


Abbildung 8.3: Latenz-Alert im Zustand *Firing* nach 6 Minuten. Die Annotations enthalten Symptombeschreibung, Prüfpfad und Runbook-Link. Quelle: Eigene Darstellung (Screenshot, Staging-Server).

Der Fehler-Alert konnte zuverlässig ausgelöst werden. Die Rate-Berechnung lieferte allerdings bei kurzen Betrachtungszeiträumen keine konsistenten Werte, vermutlich aufgrund einer Diskrepanz zwischen dem dynamisch berechneten Abfragefenster und der Erfassungssemantik der OTel-basierten Metriken. Der Sättigungs-Alert wurde nicht als eigenständiges Szenario getestet, die zugrunde liegende Metrik war jedoch korrekt verfügbar.

Für den Latenz-Alert ist die Alerting-Konzeption funktional validiert. Für den Fehler-Alert zeigt sich ein Datenqualitätsproblem in der kurzfristigen Metrikdarstellung, das die konzeptionelle Validität der Regel nicht infrage stellt, aber die operative Kurzfristdiagnose einschränkt. Der Erfüllungsgrad wird als *teilweise erfüllt* eingestuft.

8.2.5 A5: Beobachtbarkeit der Infrastrukturschicht

Prüffrage: Werden Container-Ressourcen, Datenbankpool-Auslastung und Queue-Tiefen erfasst?

Die Infrastruktur-Sektion des Dashboards beantwortet diese Frage positiv. Abbildung 8.4 zeigt PostgreSQL-Verbindungszähler, RabbitMQ-Queue-Tiefen, Host-Ressourcen und den Service Graph. Während der Belastungstests stieg die Zahl aktiver PostgreSQL-Verbindungen von einem Normalniveau um 3 auf temporär über 10. Alle fünf Infrastruktur-Exporter lieferten während des gesamten Evaluationszeitraums durchgängig Metriken. Da die Metriken verfügbar und plausibel sind, nicht aber alle Indikatoren unter gezielter Last geprüft wurden, wird A5 als *weitgehend erfüllt* bewertet.



Abbildung 8.4: Infrastruktur-Sektion des Golden-Signals-Dashboards mit PostgreSQL-Verbindungen, RabbitMQ-Queue-Tiefe, Host-Ressourcen und Service Graph. Quelle: Eigene Darstellung (Screenshot, Staging-Server).

8.3 Übergreifende Bewertungsdimensionen

Stakeholder-Tauglichkeit. Für die DevOps-Perspektive ist der Prototyp funktional nutzbar: Dashboard, Alerting-Regeln und Runbooks liefern eine zusammenhängende Diagnoseinfrastruktur. Für die Entwicklungsperspektive ermöglicht Grafana Explore die direkte Abfrage von Traces, Logs und Metriken mit bidirektionaler Korrelation, ein dediziertes Entwickler-Dashboard wurde jedoch nicht umgesetzt. Für die Product-Owner-Perspektive fehlt im Prototyp eine eigene Sicht, die technisch aus den vorhandenen Datenquellen ableitbar wäre.

Abdeckung der Risikopunkte. Von den sechs identifizierten Risikopunkten werden R2 (DB-Contention), R3 (Intransparenz ohne Tracing) und R5 (Ressourcenverdrängung) durch den Prototyp direkt adressiert. R3 wurde durch die Einführung von Distributed Tracing beseitigt. R1 (fehlende Timeouts) und R4 (Backend als Single Point of Failure) bleiben als architektonische Defizite bestehen, die über Beobachtbarkeit sichtbar, aber nicht lösbar sind.

Umsetzungsrealismus. Die Gesamtdauer von der initialen Stack-Konfiguration bis zum funktionierenden Dashboard betrug wenige Arbeitstage. Der laufende Betrieb des Observability-Stacks erfordert allerdings ein eigenes Ressourcenbudget, das bei der Kapazitätsplanung des Hosts berücksichtigt werden muss.

8.4 Grenzen und methodische Einschränkungen

Die Evaluation unterliegt fünf Einschränkungen.

Erstens, unvollständige Dienstabdeckung. Backend und WS-Gateway sind instrumentiert, AI Provider und Frontend nicht. Die Bewertung von A1 und A2 gilt nur für einen Teil des Gesamtsystems.

Zweitens, Stabilität des Observability-Stacks unter Last. Tempo wurde während der Belastungstests durch Speichererschöpfung beendet (OOM-Kill), woraufhin auch der OTel Collector den Betrieb einstellte. Infrastrukturmetriken über die Prometheus-Exporter blieben unberührt. Der Vorfall verdeutlicht, dass der Observability-Stack selbst eine Fehlerquelle darstellt, deren Auswirkungen auf einem Single-Host-System die Beobachtbarkeit des Gesamtsystems beeinträchtigen.

Drittens, eingeschränkte Aussagekraft der Rate-Berechnung. Die PromQL-Variable `$_rate_interval` lieferte bei kurzen Betrachtungszeiträumen inkonsistente Ergebnisse für die HTTP-5xx-Rate. In längeren Zeitfenstern (ab circa 12 Stunden) waren die Werte korrekt. Diese Einschränkung betrifft die Dashboard-Darstellung, nicht die Alerting-Regeln mit festen Zeitfenstern.

Viertens, fehlende Produktivdaten. Alle Tests wurden mit synthetischer Last durchgeführt. Reale Nutzungsmuster, insbesondere die Lastspitzen während Lehrveranstaltungen, konnten nicht abgebildet werden. Die Schwellenwerte müssten im Produktivbetrieb anhand realer Lastprofile kalibriert werden.

Fünftens, prototypische Testmethodik. Die Belastungsszenarien wurden manuell über Konsolenschleifen und Container-Operationen durchgeführt. Für eine belastbarere Validierung wäre ein dedizierter HTTP-Lastgenerator mit kontrollierten Lastprofilen erforderlich.

9 Diskussion

Die Evaluation in Kapitel 8 hat die Ergebnisse der prototypischen Umsetzung an den Anforderungen gemessen und funktionale Stärken wie offene Lücken identifiziert. Das vorliegende Kapitel ordnet diese Befunde in den Kontext der wissenschaftlichen Literatur ein, bewertet die Übertragbarkeit und grenzt den wissenschaftlichen vom praktischen Beitrag ab.

9.1 Einordnung der Ergebnisse im Verhältnis zur Literatur

Die Arbeit bestätigt mehrere Grundannahmen der Observability-Literatur und konkretisiert sie um operative Erkenntnisse aus der prototypischen Umsetzung.

Auf der Bestätigungsseite stehen drei Befunde. Erstens erweist sich die Kombination der drei Telemetriepfeiler als praktisch tragfähig. Erst die Verknüpfung von Span-basierten Applikationsmetriken mit Exporter-basierten Infrastrukturmetriken in einem gemeinsamen Dashboard erzeugt eine diagnostisch nutzbare Gesamtsicht (Albuquerque & Correia, 2025, S. 22). Zweitens hat sich die symptomorientierte Alarmierung bewährt. Der Latenz-Alert verhinderte in der Evaluation falsch-positive Auslösungen in Leerlaufphasen, indem er neben der p95-Überschreitung eine Mindest-Traffic-Rate als Vorbedingung verlangt. Drittens ermöglicht die Auto-Instrumentierung über den OTel Java Agent einen niedrigschwelligen Einstieg in Distributed Tracing ohne Code-änderungen.

Neben diesen Bestätigungen macht die Arbeit drei Aspekte sichtbar, die in der Literatur eher abstrakt behandelt werden. Der erste betrifft die *operative Komplexität*. Der OOM-Kill von Tempo, die Netzwerkprobleme in der Docker-Compose-Topologie und das Span-Rauschen durch die Auto-Instrumentierung traten als konkrete Problemkategorien auf, die in den herangezogenen Quellen nicht in dieser Spezifik behandelt werden. Niedermaier et al. (2019, S. 9) identifizieren zwar knappe Ressourcen und reaktive Implementierung als typische Herausforderungen, adressieren aber nicht die Kaskadeneffekte, die auf einem Single-Host-System durch den Observability-Stack selbst entstehen können.

Der zweite Aspekt betrifft die *Signalqualität*. Statt Sampling, das relevante Daten anteilig verlieren kann, setzt die Arbeit gezieltes Filtering auf Collector-Ebene ein, das diagnostisch wertlose Spans vor der Speicherung verwirft. Dieser Ansatz adressiert ein Problem, das die Literatur unter dem Stichwort Sampling diskutiert (Albuquerque & Correia, 2025, S. 10), ohne eine inhaltlich begründete Selektion auf Collector-Ebene als Lösungsstrategie vorzuschlagen.

Der dritte Aspekt betrifft die *Validierbarkeit von Alerting-Regeln*. Die Evaluation zeigt, dass die Fehler-Metriken in kurzen Zeitfenstern keine konsistent aktuellen Werte lieferten. Die Literatur benennt die Notwendigkeit des Testens (Vinnakota & Kolla, 2025, S. 175–176), beleuchtet aber die konkreten Schwierigkeiten, die sich aus dem Zusammenspiel von Metriksemantik, Query-Verhalten und Staging-Verifikation ergeben, nur begrenzt.

9.2 Übertragbarkeit auf ähnliche cloud-native Plattformen

Hoch übertragbar sind drei methodische Elemente: die Signal-Matrix-Methodik, die für jeden Dienst systematisch die vier Golden Signals auf Messgrößen, Zwecke und Rollen abbildet, die Fünf-Felder-Runbook-Struktur sowie die symptomorientierte Alert-Logik, die nutzerwirksame Zustände als Auslöser verwendet.

Bedingt übertragbar ist der OTEL-Agent-basierte Instrumentierungsansatz. Der Java Agent funktioniert besonders reibungsarm in JVM-basierten Umgebungen. Für andere Laufzeitumgebungen kann der Aufwand höher ausfallen (Bissig, 2024, S. 52). Das Grundprinzip der herstellernerneutralen Entkopplung von Instrumentierung und Backend bleibt jedoch unabhängig von der Laufzeitumgebung gültig.

Kontextabhängig sind die konkreten Implementierungsentscheidungen. Der LGTM-Stack hat sich für das Single-Host-Szenario bewährt, stellt aber keine universelle Empfehlung dar. Die Docker-Compose-Topologie und die konkreten Schwellenwerte der Alerting-Regeln sind systemspezifisch und müssen aus den jeweiligen Lastprofilen abgeleitet werden.

9.3 Wissenschaftlicher und praktischer Beitrag

Der *wissenschaftliche Beitrag* besteht in der methodisch transparenten Ableitungslogik, die von der Systemanalyse über die servicebezogene Operationalisierung der Golden Signals bis zur kriteriengestützten Evaluation reicht. Dieses Vorgehen verbindet mehrere in der Literatur häufig getrennt behandelte Konzepte zu einem integrierten Strategieentwurf. Darüber hinaus liefert die Arbeit fallbezogene operative Hinweise auf Problemkategorien, etwa Ressourcenkonflikte des Observability-Stacks mit der beobachteten Anwendung, die in der vorwiegend konzeptionell argumentierenden Literatur wenig Raum einnehmen.

Der *praktische Beitrag* umfasst eine funktionsfähige Observability-Infrastruktur für frag.jetzt aus Instrumentierungskonfigurationen, Dashboard-Definitionen, Alerting-Regeln und Runbooks. Die Signal-Matrix und die Runbook-Struktur stehen als wiederverwendbare Artefakte zur Verfügung, die auch ohne den spezifischen Technologie-Stack adaptiert werden können.

Literaturverzeichnis

- Albuquerque, C., & Correia, F. F. (2025). Tracing and Metrics Design Patterns for Monitoring Cloud-native Applications. *arXiv preprint arXiv:2510.02991*. <https://doi.org/10.48550/arXiv.2510.02991>
- Banerjee, A., & Singh, R. (2024). A Comparative Analysis of System Monitoring Architecture: Evaluating Prometheus, ELK Stack, and Custom Dashboards for Performance and Scalability. *International Journal of Computer Applications*. <https://doi.org/10.21275/SR25917095133>
- Bissig, N. (2024). *Unified Application Observability in Heterogeneous Distributed Systems* [Diss., Hochschule für angewandte Wissenschaften München]. https://opus4.kobv.de/opus4-hm/frontdoor/deliver/index/docId/590/file/Bissig_2024_MA.pdf
- Borges, M. C., Bauer, J., Werner, S., Gebauer, M., & Tai, S. (2024). Informed and assessable observability design decisions in cloud-native microservice applications. *2024 IEEE 21st International Conference on Software Architecture (ICSA)*, 69–78. <https://doi.org/10.1109/ICSA59870.2024.00015>
- Dasari, H. (2025). SITE RELIABILITY ENGINEERING PRACTICES FOR ERROR BUDGET MANAGEMENT IN LARGE-SCALE SYSTEMS. *International Journal of Applied Mathematics*, 38(5s), 991–1001. <https://doi.org/10.12732/ijam.v38i5s.366>
- Elastic. (n. d.). *Use OpenTelemetry with Elastic APM*. Elastic Docs. (besucht am 28. März 2026) <https://www.elastic.co/docs/solutions/observability/apm/opentelemetry>
- Elastic. (2024). *FAQ on Software Licensing*. Elastic. (besucht am 28. März 2026) <https://www.elastic.co/pricing/faq/licensing>
- Ewaschuk, R. (2017). Monitoring Distributed Systems [Kapitel 6 in *Site Reliability Engineering*, herausgegeben von Betssy Beyer, Google].
- Faseeha, U., Syed, H. J., Samad, F., Zehra, S., & Ahmed, H. (2025). Observability in Microservices: An In-Depth Exploration of Frameworks, Challenges, and Deployment Paradigms. *IEEE Access*, 13, 72011–72035. <https://doi.org/10.1109/ACCESS.2025.3562125>
- Fischer, S., Urbanke, P., Ramler, R., Steidl, M., & Felderer, M. (2024). An Overview of Microservice-Based Systems Used for Evaluation in Testing and Monitoring: A Systematic Mapping Study. <https://doi.org/10.1145/3644032.3644445>
- Giamattei, L., Guerriero, A., Pietrantuono, R., Russo, S., Malavolta, I., Islam, T., Dinga, M., Koziolk, A., Singh, S., Armbruster, M., et al. (2024). Monitoring tools for DevOps and microservices: A systematic grey literature review. *Journal of Systems and Software*, 208, 111906. <https://doi.org/10.1016/j.jss.2023.111906>
- Grafana Labs. (n. d. a). *Introduction to Grafana Tempo*. Grafana Tempo documentation. (besucht am 28. März 2026) <https://grafana.com/docs/tempo/latest/set-up-for-tracing/>
- Grafana Labs. (n. d. b). *Understand labels*. Grafana Loki documentation. (besucht am 28. März 2026) <https://grafana.com/docs/loki/latest/get-started/labels/>

- Gudimetla, S. (2025). Transitioning to Open-Source Observability: A Cost-Benefit Analysis and Migration Framework. *Journal of Computer Science and Technology Studies*, 7(12), 495–512. <https://doi.org/10.32996/jcsts.2025.7.12.55>
- Jalalvand, F., Baruwat Chhetri, M., Nepal, S., & Paris, C. (2024). Alert prioritisation in security operations centres: A systematic survey on criteria and methods. *ACM Computing Surveys*, 57(2), 1–36. <https://doi.org/10.1145/3695462>
- Kosińska, J., Baliś, B., Konieczny, M., Malawski, M., & Zieliński, S. (2023). Toward the observability of cloud-native applications: The overview of the state-of-the-art. *IEEE Access*, 11, 73036–73052. <https://doi.org/10.1109/ACCESS.2023.3281860>
- Li, B., Peng, X., Xiang, Q., Wang, H., Xie, T., Sun, J., & Liu, X. (2022). Enjoy your observability: an industrial survey of microservice tracing and analysis. *Empirical Software Engineering*, 27(1), 25. <https://doi.org/10.1007/s10664-021-10063-9>
- Niedermaier, S., Koetter, F., Freymann, A., & Wagner, S. (2019). On Observability and Monitoring of Distributed Systems – An Industry Interview Study. In *Service-Oriented Computing* (S. 36–52). Springer International Publishing. https://doi.org/10.1007/978-3-030-33702-5_3
- Sakinala, K. (2025). Monitoring and Observability for Cloud-Native Applications. *Journal of Computer Science and Technology Studies*, 7(8), 101–115. <https://al-kindipublishers.org/index.php/jcsts/article/view/10459>
- Soldani, J., & Brogi, A. (2022). Anomaly Detection and Failure Root Cause Analysis in (Micro) Service-Based Cloud Applications: A Survey. *ACM Comput. Surv.*, 55(3). <https://doi.org/10.1145/3501297>
- Sundström, J. (2025). Finding Faults Faster: Improving Error Tracing in Reactive Monitored Systems with Distributed Tracing Tools. (besucht am 28. März 2026) <https://www.diva-portal.org/smash/record.jsf?dswid=1070&pid=diva2%3A1964604>
- TH Mittelhessen & arsnova. (2025). *frag.jetzt – Open-Source Q&A-Plattform*. (besucht am 28. März 2026) <https://gitlab.arsnova.eu/arsnova/frag.jetzt/-/blob/staging/README.md>
- Usman, M., Ferlin, S., Brunstrom, A., & Taheri, J. (2022). A survey on observability of distributed edge & container-based microservices. *IEEE Access*, 10, 86904–86919. <https://doi.org/10.1109/ACCESS.2022.3193102>
- Vinnakota, K., & Kolla, M. (2025). Creating Effective Alerts for Monitoring Distributed Systems. *International Journal of Computer Trends and Technology*, 73, 172–178. <https://doi.org/10.14445/22312803/IJCTT-V73I5P122>

Index

Four Golden Signals, 1
frag.jetzt, 1

Observability, 1

A Verzeichnis der KI-Nutzung

Erklärung zur Nutzung KI-basierter Werkzeuge

Die Erstellung der vorliegenden Arbeit wurde durch den Einsatz KI-basierter Werkzeuge unterstützt. Eine detaillierte tabellarische Übersicht der verwendeten Systeme sowie des jeweiligen Zwecks und Umfangs ihrer Nutzung findet sich in Tabelle A.1.

Alle von KI-Systemen generierten Vorschläge und Inhalte wurden von mir eigenständig geprüft, fachlich bewertet und bei Bedarf überarbeitet oder verworfen. Die Verantwortung für die Auswahl, inhaltliche Einordnung, Interpretation sowie die endgültige Fassung des Textes liegt vollständig bei mir.

Grundlage für den Umgang mit den Werkzeugen bildet die Richtlinie für die Nutzung von KI im Studium der IU Internationale Hochschule.

Declaration on the Use of AI-based Tools

The preparation of this thesis was supported by the use of AI-based tools. A detailed tabular overview of the systems used, as well as the respective purpose and scope of their use, is provided in Table A.1.

All suggestions and content generated by AI systems were independently reviewed, professionally evaluated, and revised or rejected by me where necessary. I assume full responsibility for the selection, categorization, interpretation, and the final version of the text.

The handling of these tools follows the current recommendations of the Guidelines for the Use of AI in Academic Studies at IU International University.

Tabelle A.1 listet alle in dieser Arbeit verwendeten KI-Werkzeuge auf.

A.1 Copilot Prompts für die unterstützende Systemanalyse von frag.jetzt

- 1 Analysiere die Projektstruktur von frag.jetzt. Welche Module, Services und Hauptkomponenten gibt es? Erstelle eine tabellarische Uebersicht mit Modulname, Technologie/Framework und Hauptverantwortung.
- 2 Identifiziere alle externen HTTP-Aufrufe (REST-Clients, WebClients, Feign-Clients) im Spring-Boot-Backend. Welche externen Dienste werden aufgerufen, mit welchen Endpunkten, und gibt es Timeout- oder Retry-Konfigurationen?
- 3 Liste alle REST-Controller im Backend auf. Gruppiere sie nach fachlicher Funktion (z. B. Room-Management, Question-Management, Auth, AI-Service) und notiere die HTTP-Methoden und Pfade.
- 4 Gibt es bereits Observability-bezogene Konfigurationen? Suche nach: Micrometer, Actuator, OpenTelemetry, Prometheus, Logging-Frameworks (Logback, Log4j), Health-Check-Endpoints, Tracing-Bibliotheken.
- 5 Analysiere die Datenbankschicht. Welche Entities/Tabellen gibt es, welche Repositories werden verwendet, und gibt es komplexe Queries oder Stored Procedures, die als Performancerisiko gelten koennten?
- 6 Beschreibe die WebSocket-Kommunikation. Welche STOMP-Topics oder Channels gibt es, welche Nachrichten werden darueber ausgetauscht, und wie wird die Verbindung verwaltet?
- 7 Analysiere die Deployment-Konfiguration (docker-compose.yml, Kubernetes-Manifeste, CI/CD-Pipeline). Welche Container/Services werden deployt, wie sind sie vernetzt, und welche Ressourcenlimits sind konfiguriert?

Listing A.1: Copilot-Prompts zur Architekturanalyse

Tabelle A.1: Verzeichnis der in dieser Arbeit genutzten KI-Werkzeuge. Quelle: Eigene Darstellung.				
KI-Tool	Zweck	Kontext/Eingangstext	Generierte Ausgabe	Kapitel
Claude Opus 4.6	Formatierung und Formulierungsverbesserung der Runbooks	Strukturierung des methodischen Ansatzes	Vorschlag zur Gliederung der Methodik	Kap. 3.1
Claude Opus 4.6	Code-Refactoring	Optimierung bestehender Algorithmen	Vorschläge zur Effizienzverbesserung	Kap. 4.3
GitHub Copilot	Code-Vervollständigung	Hilfe bei Instrumentierung und Deployment der Observability-Komponenten	Vorschläge für Funktionsimplementierung	Kap. 4.2
Gemini 3 Pro	Literaturrecherche	Zusammenfassung aktueller Forschungsarbeiten	Überblick über Stand der Technik	Kap. 2.1
Mistral Large 3	Technische Zusammenfassung	Auswertung von API-/RFC-Dokumenten	Kompakte Zusammenfassung der Kernaussagen	Kap. 2.4
OpenAI GPT-5.2	Formulierungsverbesserungen	Überarbeitung der Einleitung	Alternative Formulierungen zur Problemstellung	Kap. 1.2
OpenAI GPT-5.3	Hilfe bei Dashboard Design	Vorschlag zu möglichen Tool-Stacks	Stichpunkt-Zusammenfassung mit Quellenhinweisen	Kap. 2.3

Bei KI-Tools genau sagen, welches Modell / Version oder ähnliches — frag KI wie man die KI referenziert.

A.2 C4-Container-Diagramm in Vollansicht

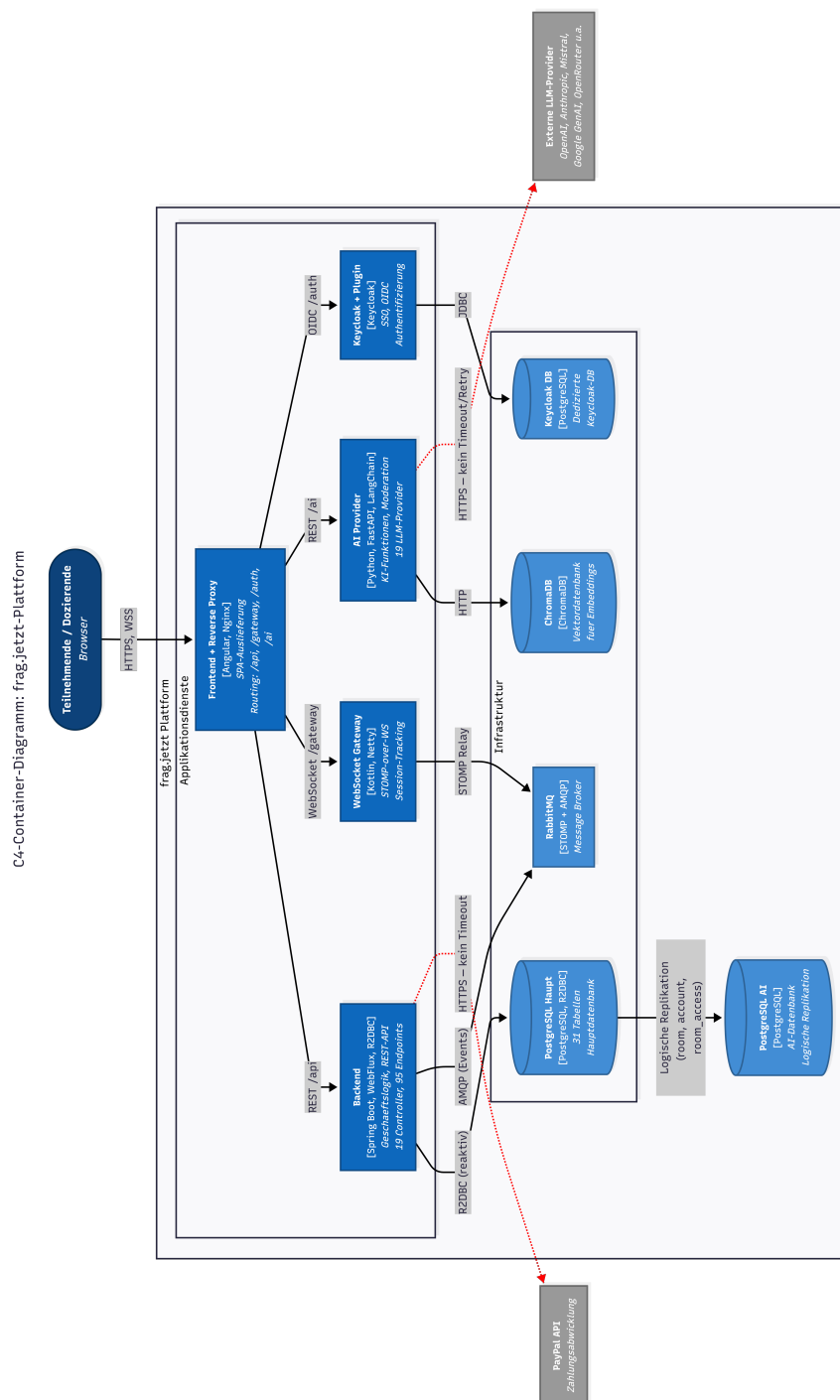


Abbildung A.1: C4-Container-Diagramm der frag.jetzt-Architektur in Vollseitenansicht. Quelle: Eigene Darstellung.

Erklärung der Selbstständigkeit

Hiermit versichere ich, die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt zu haben.

Frankfurt am Main, den 28. März 2026

Max Mustermann